

STRUCTURED QUERY LANGUAGE (SQL)

Historical Note

The origins of SQL can be traced to an IBM research paper published in 1974, titled *SEQUEL: A Structured English Query Language*, by Donald D. Chamberlin and Raymond F. Boyce. Raymond Boyce, who sadly died in the same year, 1974, was the person whose name is enshrined in *Boyce-Codd Normal Form* (BCNF), described in Chapter 7 of the theory book. This paper actually said nothing about database management but it assumed the existence of relation-like things called tables and defined a notation for expressing queries on tables in the declarative manner that is expected for relational databases.

A prototype implementation of SEQUEL, named SEQUEL-XRM, was produced in 1974–75. Experience with this prototype led in 1976–77 to a revised version of the language, SEQUEL/2, subsequently renamed SQL for legal reasons. Work then began on System R, another, more ambitious prototype, implementing a subset of that revised version.

A significant aim of System R was to gainsay those pundits who were sceptical about the possibility of providing a relational DBMS with performance comparable to that obtainable with conventional database technology at the time, and it was widely lauded as having succeeded in that aim. System R became operational in 1977 and was installed at a number of internal sites in IBM and at a few selected customer sites. Experiences with System R resulted in further changes to SQL and soon expectations were high that IBM would eventually develop commercial SQL products—so high that one vendor organization, now named Oracle Corporation, beat IBM to it in 1979 with the first release of ORACLE. IBM's own first SQL product, SQL/DS for the VSE environment, appeared in 1981 and implementations on other platforms followed during the 1980s. One of these was DB2 for OS/MVS in 1983, and much later DB2 became the generic name for all of IBM's implementations.

A host of other vendors joined the game over the next two decades, including, etc., Microsoft (with SQL Server, naughtily pronounced “sequel server”!), Sybase, Ingres, and Digital Equipment Corp., (with RDB, now owned by Oracle). Those last two, Ingres and RDB, originally used their own languages, both inspired by Codd's relational calculus, but later acquired an SQL interface.

The first edition of the international (ISO) standard for SQL appeared in 1987 as a ratification of one produced a year earlier by the Database Committee X3H2 of the American National Standards Institute (ANSI). Further development of the standard has taken place since then in ISO—or rather, under the auspices of a “joint technical committee”, JTC 1, of ISO and IEC, responsible for all international IT standards. The editions are referred to as SQLyyyy, whereyyyy is the year of publication.

The so-called “referential integrity” feature, supporting primary keys and foreign keys, appeared in SQL:1989. SQL:1992 contained so many additional features that its language had been referred to at the time as SQL/2. In 1996 additional features known as SQL/PSM (a programming language for stored procedures and functions) and SQL/CLI (a “call level interface”) were added. At the same time, the committee was working on further extensions to the language, resulting in the appearance of “SQL/3” as SQL: 1999, whose most significant addition was the so-called “object-relational” feature supporting user defined types. Further revisions appeared in 2003, 2007, and 2011. That last one, SQL: 2011 (reference [15]), is notable for its modicum of support for “temporal database” management.

Many of the additional features appearing in post-1989 editions are specified as optional, meaning that they aren't required for an implementation to qualify as standard-conforming.

Historical notes appearing later in this book, concerning specific features of SQL, refer mostly to the pre-2000 editions of the ISO SQL standard: 1987, 1989, 1992, 1996, 1999, though in connection with the notation for queries can sometimes take you right back to the 1974 SEQUEL paper.

Types of database Management Systems:

Basically, there are two types of DBMSs:

Relational and Non-relational, also referred to as SQL and NoSQL respectively.

Classification of Databases:

Centralized database	Cloud database	Commercial database
Distributed database.		
End-user database	Graph database	NoSQL database
database.		Object-oriented

The NoSQL Database.

NoSQL originally means a non-relational database (or non-SQL). It provides a data storage and recovery mechanism. ...

As of August 2022, the most popular database management system (DBMS) in the world was Oracle, MySQL and Microsoft SQL server rounded out the top three.

The Top 10 Popular database management systems are:

MySQL, MariaDB, Microsoft SQL Server, Oracle DBMS, PostgreSQL, MongoDB, Redis, IBM DB2, Elasticsearch, SQLite

The RDBMS include:

MySQL, image of Microsoft SQL Server, Microsoft SQL Server, Oracle Database, PostgreSQL, IBM Db2, SQLite, MariaDB, Microsoft Access, MongoDB, SAP HANA, IBM Informix, Microsoft Azure SQL Database, Neo4j, SQL Anywhere, Firebird, CUBRID, SQL Server Express, Apache Hive, and solidDB.

The NoSQL databases:

image of MongoDB, MongoDB, Apache Cassandra, Redis, Neo4j, CouchDB, Amazon DynamoDB, Apache HBase, OrientDB, Elasticsearch, RavenDB,

MySQL

MySQL is a high-speed data processing and data productivity tool with comprehensive features. It is designed to increase the security and scalability of databases, provides a reliably cost-effective tool, technical support and counteracts potential risks. High-volume business sites can deploy business-oriented complex MySQL applications.

MySQL is an open-source relational database which runs on a number of different platforms such as Windows, Linux, and Mac OS, etc., interfaces with languages such: C, Java, SQL, C++, Perl, Python, Tcl, and PHP. It has a cloud version and can be used for packaged software, and business-critical systems and high-volume websites.

As at March, 2025 the latest MySQL community Edition is MySQL 8.4.3 (LTS Release) released on October 15, 2024. You can download the MySQL installer for windows from the MySQL Download page.

Features

It is easy to learn the foundational features without a programming background. The open-source nature grant users complete freedom to customize data. It is Well-suited for small businesses and entrepreneurs because of the low-cost structure, compatible with up-to-date industry's DBMS practices, provides instant integration with Apache web engines and the data development process is adjustable for small and heavy applications.

Cons

Queries get stuck even after a refresh or restart
There is overdependence on third-party add-ons
Data operations in Linux OS can get complicated.

MySQL Community Edition is the freely downloadable version of the world's most popular open source database. It is available under the GPL license and is supported by a huge and active community of open source developers

Releases:

Latest LTS Release: MySQL 8.4.3 (LTS Release) was released on October 15, 2024.

Download: You can download the MySQL Installer for Windows from the MySQL Downloads page.

MySQL 8.0: MySQL 8.0.40 is also a general availability release, released on October 15, 2024.

MySQL 9.1: MySQL 9.1.0 is an innovation release, released on October 15, 2024.

MySQL Workbench: MySQL Workbench is a cross-platform software powered by Oracle, which lets you perform development with MySQL Databases.

MySQL Community Edition: It is a freely downloadable version of the world's most popular open-source database.

MySQL Installer 8.0.41

Note: MySQL 8.0 is the final series with MySQL Installer. As of MySQL 8.1, use a MySQL product's MSI or Zip archive for installation. MySQL Server 8.1 and higher also bundle MySQL Configurator, a tool that helps configure MySQL Server.

An SQL developer must decide what type of data that will be stored inside each column when creating a table. The data type is a guideline for SQL to understand what type of data is expected inside of each column, and it also identifies how SQL will interact with the stored data.

Note: Data types might have different names in different database. And even if the name is the same, the size and other details may be different! Always check the documentation!

MySQL Data Types (Version 8.0)

In MySQL there are three main data types: String, Numeric, and Date/time.

The String Data Types

Data type	Description
• CHAR(size)	A FIXED length string (can contain letters, numbers, and special characters). The size parameter specifies the column length in characters - can be from 0 to 255. Default is 1
• VARCHAR(size)	A VARIABLE length string (can contain letters, numbers, and special characters). The size parameter specifies the maximum string length in characters - can be from 0 to 65535
• BINARY(size)	Equal to CHAR(), but stores binary byte strings. The size parameter specifies the column length in bytes. Default is 1
• VARBINARY(size)	Equal to VARCHAR(), but stores binary byte strings. The size parameter specifies the maximum column length in bytes.
• TINYBLOB	For BLOBs (Binary Large Objects). Max length: 255 bytes
• TINYTEXT	Holds a string with a maximum length of 255 characters
• TEXT(size)	Holds a string with a maximum length of 65,535 bytes
• BLOB(size)	For BLOBs (Binary Large Objects). Holds up to 65,535 bytes of data
• MEDIUMTEXT	Holds a string with a maximum length of 16,777,215 characters
• MEDIUMBLOB	For BLOBs (Binary Large Objects). Holds up to 16,777,215 bytes of data
• LONGTEXT	Holds a string with a maximum length of 4,294,967,295 characters
• LONGBLOB	For BLOBs (Binary Large Objects). Holds up to 4,294,967,295 bytes of data
• ENUM(val1, val2, val3, ...)	A string object that can have only one value, chosen from a list of possible values. You can list up to 65535 values in an ENUM list. If a value is inserted that is not in the list, a blank value will be inserted. The values are sorted in the order you enter them
• SET(val1, val2, val3, ...)	A string object that can have 0 or more values, chosen from a list of possible values. You can list up to 64 values in a SET list

Numeric Data Types

Data type	Description
• BIT(size)	A bit-value type. The number of bits per value is specified in size. The size parameter can hold a value from 1 to 64. The default value for size is 1.

- **TINYINT(size)** A very small integer. Signed range is from -128 to 127. Unsigned range is from 0 to 255. The size parameter specifies the maximum display width (which is 255)
- **BOOL** Zero is considered as false, nonzero values are considered as true.
- **BOOLEAN** Equal to **BOOL**
- **SMALLINT(size)** A small integer. Signed range is from -32768 to 32767. Unsigned range is from 0 to 65535. The size parameter specifies the maximum display width (which is 255)
- **MEDIUMINT(size)** A medium integer. Signed range is from -8388608 to 8388607. Unsigned range is from 0 to 16777215. The size parameter specifies the maximum display width (which is 255)
- **INT(size)** A medium integer. Signed range is from -2147483648 to 2147483647. Unsigned range is from 0 to 4294967295. The size parameter specifies the maximum display width (which is 255)
- **INTEGER(size)** Equal to **INT(size)**
- **BIGINT(size)** A large integer. Signed range is from -9223372036854775808 to 9223372036854775807. Unsigned range is from 0 to 18446744073709551615. The size parameter specifies the maximum display width (which is 255)
- **FLOAT(size, d)** A floating point number. The total number of digits is specified in size. The number of digits after the decimal point is specified in the d parameter. This syntax is deprecated in MySQL 8.0.17, and it will be removed in future MySQL versions
- **FLOAT(p)** A floating point number. MySQL uses the p value to determine whether to use **FLOAT** or **DOUBLE** for the resulting data type. If p is from 0 to 24, the data type becomes **FLOAT()**. If p is from 25 to 53, the data type becomes **DOUBLE()**
- **DOUBLE(size, d)** A normal-size floating point number. The total number of digits is specified in size. The number of digits after the decimal point is specified in the d parameter
- **DOUBLE PRECISION(size, d)**
- **DECIMAL(size, d)** An exact fixed-point number. The total number of digits is specified in size. The number of digits after the decimal point is specified in the d parameter. The maximum number for size is 65. The maximum number for d is 30. The default value for size is 10. The default value for d is 0.
- **DEC(size, d)** Equal to **DECIMAL(size,d)**

Note: All the numeric data types may have an extra option: **UNSIGNED** or **ZEROFILL**. If you add the **UNSIGNED** option, MySQL disallows negative values for the column. If you add the **ZEROFILL** option, MySQL automatically also adds the **UNSIGNED** attribute to the column.

Date and Time Data Types

- | Data type | Description |
|-----------------------|--|
| DATE | A date. Format: YYYY-MM-DD. The supported range is from '1000-01-01' to '9999-12-31' |
| DATETIME(fsp) | A date and time combination. Format: YYYY-MM-DD hh:mm:ss. The supported range is from '1000-01-01 00:00:00' to '9999-12-31 23:59:59'. Adding DEFAULT and ON UPDATE in the column definition to get automatic initialization and updating to the current date and time |
| TIMESTAMP(fsp) | A timestamp. TIMESTAMP values are stored as the number of seconds since the Unix epoch ('1970-01-01 00:00:00' UTC). Format: YYYY-MM-DD hh:mm:ss. The supported range is from '1970-01-01 00:00:01' UTC to '2038-01-09 03:14:07' UTC. Automatic initialization and updating to the current date and time can be specified using DEFAULT CURRENT_TIMESTAMP and ON UPDATE CURRENT_TIMESTAMP in the column definition |
| TIME(fsp) | A time. Format: hh:mm:ss. The supported range is from '-838:59:59' to '838:59:59' |
| YEAR | A year in four-digit format. Values allowed in four-digit format: 1901 to 2155, and 0000. |

Note: MySQL 8.0 does not support year in two-digit format.

What Is an SQL Database?

An SQL database is one whose symbols are organized into a collection of *tables*.

SQL Is a Database Language

The commands given to a DBMS by an application are written in the database language of the DBMS. The term *data sublanguage* is sometimes used instead of database language. The sub-prefix refers to the fact that application programs are sometimes written in some more general-purpose programming language (the "host" language), in which the database language commands are embedded in some prescribed style. Sometimes the embedding style is such that the embedded statements are unrecognized by the host language compiler or interpreter, and some special *pre-processor* is used to replace the embedded statements by, for example, CALL statements in the host language.

Why SQL?

Until recently, if you weren't working on a large database system, you probably had only a passing knowledge of SQL. With the advent of client/server development tools (such as Visual Basic, Visual C++, ODBC, Borland's Delphi, and Powersoft's PowerBuilder) and the movement of several large databases (Oracle and Sybase) to the PC platform, most business applications being developed today require a working knowledge of SQL.

What Does an SQL DBMS Do?

In response to requests from application programs, an SQL DBMS can:

- o create and destroy base tables
- o Change system security settings
- o Add user permissions on databases or tables
- o take note of integrity rules expressed as *constraints*
- o take note of *authorisations* (who is allowed to do what, to what)
- o update variables (honouring constraints and authorisations)
- o provide results of *queries*

In the remaining sections of this chapter you will see examples of how an SQL DBMS does these things (via commands—usually called statements—written in SQL, of course).

SQL in Application Programming

SQL was originally made an ANSI standard in 1986. The ANSI 1989 standard (often called SQL-89) defines three types of interfacing to SQL within an application program:

Approaches

- Embedded SQL
 - Older approach (since the 70s)
- Call Level Interface (CLI)
 - Most recent
 - SQL/CLI, ODBC, JDBC

Embedded SQL

- In the embedded SQL approach the programmer inserts SQL statements directly in the source code of the host programming language
- A precompiler is used to translate the code so that SQL statements are translated into function/procedure calls of the specific DBMS API
- From a file containing embedded SQL to a file in the same language containing function calls

Shared Variables

- To connect SQL and the host-language program, the two parts must share some variables.
- Declarations of shared variables are bracketed by:


```
EXEC SQL BEGIN DECLARE SECTION;
<host-language declarations>
EXEC SQL END DECLARE SECTION;
```

- In SQL, the shared variables must be preceded by a colon.
 - They may be used as constants provided by the host-language program.
 - They may get values from SQL statements and pass those values to the host-language program.
- In the host language, shared variables behave like any other variable.

In Call Level Interface

- Sending commands to DBMS by means of function calls of an API
 - standard **SQL/CLI** ('95 and then part of SQL:1999)
 - **ODBC**: proprietary (Microsoft) implementation of SQL/CLI
 - OLE DB: high level API
 - ADO: higher level API
 - **JDBC**: CLI for Java

Before the concept of dynamic SQL evolved, embedded SQL was the most popular way to use SQL within a program. Embedded SQL, which is still used, uses *static* SQL (the SQL statement is compiled into the application and cannot be changed at runtime). The principle is much the same as a compiler versus an interpreter. The performance for this type of SQL is good; however, it is not flexible and cannot always meet the needs of today's changing business environments.

The ANSI 1992 standard (SQL-92) extended the language and became an international standard. It defines three levels of SQL compliance: entry, intermediate, and full. The new features introduced include the following:

- Connections to databases
- Scrollable cursors
- Dynamic SQL
- Outer joins

Dynamic SQL allows you to prepare the SQL statement at runtime. Although the performance for this type of SQL is not as good as that of embedded SQL, it provides the application developer (and user) with a great degree of flexibility. A call-level interface, such as ODBC or Sybase's DB-Library, is an example of dynamic SQL.

Call-level interfaces should not be a new concept to application programmers. When using ODBC, for instance, you simply fill a variable with your SQL statement and call the function to send the SQL statement to the database. Errors or results can be returned to the program through the use of other function calls designed for those purposes. Results are returned through a process known as the *binding of variables*.

Classification of SQL Statements

	Keywords/Commands	Function	Remarks
1	CREATE, ALTER, DROP, RENAME TRUNCATE	Data Definition (DD)	These set, changes and removes table structures.
2	INSERT, UPDATE, DELETE, MERGE	Data Manipulation (DM)	These are used to manipulate existing records.
3	SELECT, FROM	Data Retrieval (DR)	This retrieve data from tables and views of DBs
4	COMMIT, ROLLBACK SAVEPOINT	Transaction Control (TC)	Manages the changes made by DM.
5	GRANT, REVOKE	Data control (DC)	Gives or removes right to both dbase and structures.

Creating and Destroying Base Tables

SQL Create Table Statement- The create table statement is used to create a new table.

The CREATE TABLE statement defines a table. The definition must include its name and the names of attributes of its columns. The definition can include other attributes of the table, such as its primary key (or check constraints). This statement comes under the DDL statement.

Syntax:

```
SQL> Create table tablename
(Column1 data type [default exp] [Constraint],
Column2 data type [default exp] [Constraint],
Column3 data type [default exp] [Constraint],
);
```

To create a new table, enter the keywords create table followed by the table name, followed by an open parenthesis, followed by the first column name, followed by the data type for that column, followed by any optional constraints, and followed by a closing parenthesis. It is important to make sure you use an open parenthesis before the beginning table and a closing parenthesis after the end of the last column definition. Make sure you separate each column definition with a comma. For Oracle DB all SQL statements should end with a semi-colon ";".

Example:

```
Create table employee (
    Name varchar2 (15),
    Eid number (6),
    Age number (3),
    City varchar2 (30),
    State varchar2 (20)
);
```

If you want to see the structure of the table then use the following command on SQL prompt.

```
DESC employee;
```

The above command displays the entire attribute names with their data type and if constraints are imposed then their detail also.

You can also create a table from an existing table by copying the existing table's columns. It is important to note that when creating a table in this way, the new table will be populated with the records from the existing table (based on the SELECT Statement).

Syntax #1 - Copying all columns from another table

```
CREATE TABLE new _table
AS (SELECT * FROM old_table);
```

Example:

```
CREATE TABLE persons
AS (select * from persons);
```

The command would create a new table called persons that included all columns from the persons table.

If there were records in the old table, then the new table would also contain the records selected by the SELECT statement.

Syntax #2 - Copying Selected columns from another table

```
Create Table Tablename
(<columnname1>" columnname2>) as select
```

Statement;

Where <columnname1>, columnname2> are the new columns in new table with new name to be associated with the values returned by the subquery.

Example:

```
Create Table Persons
(Person_id, name)
AS SELECT (P_Id, FirstName) from persons;
```

Note: Creating a table from another table without copying any values from the old table:

Example:

```
CREATE TABLE persons
AS (select * from persons where 1=2);
```

SQL ALTER - Modify the Table Structure with alter Table Command

After creating the table, one may have need to change the table either by adding new columns or by modifying tile existing columns.

The SQL ALTER TABLE statement is the command that makes changes to the definition of an SQL table. Table structure can be changed by using alter command. With this command Field type or property can be changed or a new field can be added.

To alter a table, the table must be in your schema, or you must have alter table privilege.

After the table(s) are created and begin to be used, requirements are likely to occur which requires modifications in the structure of the table, such as adding new columns to the table, modifying and existing column's definition etc. So, these modifications can be performed using the ALTER table DDL statement.

Using ALTER TABLE statement, you can make the following modifications:

- o Add a column to a table.
- o Change the definition of an existing column in a table.
- o Enable or disable integrity constraint.
- o Specify the default value for the existing columns.
- o Add or drop one or more columns to / from existing table.
- o Change an existing column from mandatory to optional or vice versa.
- o Enable or disable integrity constraints.
- o Add/ Modify / Delete integrity constraints associated to / from the table.
- o Specify a default value for existing column.

NOTE: To ALTER a table, the table must be contained in your schema or you must have either the ALTER object privilege for the table or ALTER any table system privilege.

To Add New COLUMNS / CONSTRAINTS IN Existing Table

The syntax is:

```
ALTER TABLE <tablename>
ADD (column_specification I constrain_specification, ....
column_specification Iconstrain_specification);
```

Where tablename corresponds to the name of the table, column_specification corresponds to the valid specification for a column (columnname and datatype) and constraint specification corresponds to valid constraint specification.

Example: To add address column into the INSTRUCTOR table with column width 30 and data type varchar2 is shown below.

```
ALTER TABLE instructor
ADD (address VARCHAR2(30));
```


Instead of adding columns one by one we can add multiple columns in the table. To add address and salary columns into the INSTRUCTOR table, the statement is

```
ALTER TABLE instructor
  ADD (address VARCHAR2 (30),
       Salary NUMBER (10, 2));
```

Similarly, you can add constraints to the existing table. If you forget to add primary key constraint to the table at the time of creation, you can add it later using ALTER TABLE statement. Similarly, you can add other constraints such as foreign key, check, unique etc.

Now let us suppose that no primary key constraint exists for INSTRUCTOR table because if it exists then we cannot define a new one unless we drop the existing one, we add a primary key constraint with inst_id column of the INSTRUCTOR table, using ALTER TABLE statement given below.

```
ALTER TABLE instructor
  ADD (CONSTRAINT PK_INSTRUCTOR PRIMARY KEY( INST_ID));
```

Let us assume that no foreign key exists in the INSTRUCTOR table which references the (dept_id) primary key of DEPARTMENT table. So, we add a foreign key constraint on the dept_id column of the INSTRUCTOR table which refers to the dept_id primary key column of the DEPARTMENT table, using the ALTER TABLE command given below.

```
ALTER TABLE instructor
  ADD (CONSTRAINT FK_INSTRUCTOR FOREIGN KEY (dept_id)
       REFERENCES department (dept_id));
```

The following points should be kept in mind when adding new columns / constraints into the existing tables.

- o You can add one or more than one column at the same time.
- o There is no need of parentheses; if you adding only a single column or constraint.
- o Before adding a primary key makes sure that no duplicate values exist for a column (s) on which primary key is applied.
- o Before adding a foreign key, make sure that primary key constraint must exist in the referring table.
- o You may add a column at any time if NOT NULL is not specified. However, you can add a new column with NOT NULL only if the table is empty.
- o If a table contains records and you add a new column (s) to this table, then the initial value of each row for the new column is NULL.

Modifying Columns! Constraints In Existing Table

The ALTER TABLE statement also offers the ability to modify columns / constraints in the existing table without impacting any other data in the table. Using ALTER TABLE statement, we can increase or decrease the column widths, changing a column from mandatory to optional (NOT NULL to NULL) and vice versa.

The syntax is:

```
ALTER TABLE <tablename>
  MODIFY (column_specification I, constrain_specification, ...
         column_specification, constrain_specification);
```

To increase the width of name column in the INSTRUCTOR table from 15 to 20, use the following

ALTER TABLE statement.

```
Alter table instructor
MODIFY (name varchar2 (20));
```

Oracle doesn't allow you to decrease the column's width even if all the column values are of valid length. So to reduce the width of the column, the values must be set to NULL.

You can modify a column from NULL to NOT NULL constraint if the table contains no records in that column. However, if the column contains values for each row of the table you can not modify it from NULL to NOT NULL.

```
Alter table instructor
MODIFY (name varchar2 (20) NOT NULL);
```

You can change the data type of the columns if column contains NULL values for every row of the table.

```
Alter table instructor
MODIFY (name char (20));
```

Modify Column in a Table

We can also change the column's data type using the ALTER TABLE command with MODIFY or ALTER COLUMN clause. For example,

In SQL Server

```
ALTER TABLE Customers
ALTER COLUMN age VARCHAR(2);
```

In MySQL

```
ALTER TABLE Customers
MODIFY COLUMN age VARCHAR(2);
```

In Oracle

```
ALTER TABLE Customers
MODIFY age VARCHAR(2);
```

In PostgreSQL

```
ALTER TABLE Customers
ALTER COLUMN age TYPE VARCHAR(2);
```

Here, the SQL command changes the data type of the age column to VARCHAR in the Customers table.

Dropping Columns/Constraints

You can not only modify columns that exist in your tables but you can also drop them entirely. You can also remove the constraints from the table using the ALTER TABLE statement. When a constraint is dropped, any associated index with that constraint is also dropped.

The syntax for dropping a column is:

```
ALTER TABLE <tablename> DROP COLUMN <Columnname>;
```

This syntax is valid if you want to drop one column at a time.

To drop the PHONE column from the INSTRUCTOR table, the ALTER TABLE statement is:

SQL > Alter table instructor drop column phone;

If a user wishes to drop multiple columns using the ALTER TABLE statement, then use the following syntax.

```
ALTER TABLE <tablename>
DROP (<columnname1>,< columnname2>);
```

To drop the post and the name columns from the INSTRUCTOR table, the ALTER TABLE statement is:

```
ALTER table instructor
DROP (post, name);
```

To drop the primary key inst_id from the instructor table. Use the following ALTER TABLE statement:

```
ALTER table instructor
DROP PRIMARY KEY;
```

If you know the name of the constraint, then you can drop all types of constraints like primary key, foreign key, check constraints etc. The syntax for executing above using ALTER TABLE statement is:

```
ALTER TABLE <tablename>
DROP CONSTRAINT <constraintname>;
```

Suppose we want to drop the foreign key column DEPT_ID with constraint name FK_INSTRUCTOR then use the following statement.

```
SQL> ALTER table instructor
      DROP CONSTRAINT FK_INSTRUCTOR;
```

The following points should be kept in mind when dropping the column / constraints:

- o You cannot drop all the columns in a table.
- o You cannot drop a columns from tables whose owner is SYS.
- o You cannot drop a parent, key column (primary key) unless you drop the foreign key that references it. To do this, CASCADE keyword is used.
SQL > alter table department DROP PRIMARY KEY CASCADE;
- o You can enable or disable the key constraints in situations like when loading large amount of data into a table, performing batch operations that make massive changes to a table, migrating the organization's legacy data.

To disable constraint write:

```
Alter TABLE instructor Disable Constraint PK_INSTRUCTOR;
```

To enable constraint:

```
Alter TABLE instructor Enable Constraint PK_INSTRUCTOR;
```

Instead of dropping a column, you can make the column unused and drop it later on, It thus makes the response time faster than if your executed the DROP clause. After a column has been marked as unused, you have no access to that column. The unused columns will not be retrieved with SELECT * query. The SET UNUSED information is stored in USER_UNUSED _COL_TABS dictionary view.

```
alter table instructor SET UNUSED COLUMN phone;
```

SQL DROP TABLE Statement

In SQL, DROP TABLE is used to delete the tables in our database. For example:

`DROP TABLE my_table;`

Here, the SQL command will delete a table named my_table.

Note: make sure you have admin or DROP permission to run this command.

Note: When we delete a database table, all records within a table are also deleted.

DROP TABLE IF EXISTS

While dropping a table that does not exist, throws an error. To fix this issue, we can add an optional IF EXISTS command while dropping a table. For example,

`DROP TABLE IF EXISTS my_table;`

Here, the SQL command will only drop a table if there is one with a same name.

SQL Delete and Truncate Rows

In this tutorial, we'll learn to delete rows from a table in SQL with the help of examples.

In SQL, we use the DELETE statement to delete row(s) from a database table. For example,

`DELETE FROM Customers`

`WHERE customer_id = 5;`

Here, the SQL command will delete a row from the Customers table where customer_id is 5.

How to delete a row in SQL

Example: SQL DELETE command

Delete all Rows in a Table

The WHERE clause determines which rows to delete. However, we can delete all rows at once if we omit the WHERE clause. For example:

`DELETE FROM Customers;`

Here, the SQL command deletes all rows from a table.

Note: Be careful when you use DELETE. Records may lose permanently if the database is not backed up.

Truncate Table in SQL

The TRUNCATE TABLE clause is another way to delete all rows from a table at once. For example:

`TRUNCATE TABLE Customers;`

Here, the SQL command does exactly the same thing the above command does.

Note: The TRUNCATE clause doesn't support the WHERE clause.

Delete Vs Truncate

The main difference between both statements is that DELETE FROM statement supports WHERE clause whereas TRUNCATE does not.

That means, we can delete single or multiple rows using the DELETE FROM statement while the TRUNCATE statement deletes all records from the table at once.

We can mimic the TRUNCATE statement with DELETE FROM statement by omitting the WHERE clause. For example,

`DELETE FROM Customers;`

is similar to,

`TRUNCATE TABLE Customers;`

SQL INSERT INTO SELECT Statement

The INSERT INTO SELECT statement is used to copy records from one table to another existing table. For example,

```
INSERT INTO OldCustomers
SELECT *
FROM Customers;
```

Here, the SQL command copies all records from the Customers table to the OldCustomers table.

Note: To run this command, the

- the database must already have a table named OldCustomers
- the column names of the OldCustomers table and the Customers table must match

If we want to copy data to a new table (rather than copying in an existing table), we use the SELECT INTO statement.

Copy Selected Columns Only

We can also copy only the selected columns from one table to another. For example,

```
INSERT INTO OldCustomers(customer_id, age)
SELECT customer_id, age
FROM Customers;
Run Code
```

Here, the SQL command only copies records from the customer_id column and country column to the OldCustomers table.

Note: If there are columns other than customer_id and age in the OldCustomers table, the value of those columns will be NULL.

Copy Records Matching a Condition

We can use the WHERE clause with INSERT INTO to copy those rows that match the specified condition. For example:

```
INSERT INTO OldCustomers
SELECT *
FROM Customers
WHERE country = 'USA';
```

Here, the SQL command copies rows that are the value of the country column as USA.

Copy From two Tables to One

We can also copy records from two different tables using the JOIN clause with INSERT INTO SELECT. For example,

```
INSERT INTO OldCustomerOrders
SELECT Customers.customer_id, Customers.first_name, Orders.amount
FROM Customers
JOIN Orders
ON Customers.customer_id = Orders.customer_id;
```

Here, the SQL command copies customer_id and first_name from the Customers table and the amount from the Orders table in an existing table OldCustomerOrders. To learn more, visit SQL JOIN.

Note: If records are already present in an existing table, new rows will be appended. Columns in the existing table may throw errors such as NOT NULL Constraint Failed, UNIQUE Constraint Failed while copying data.

Avoid Duplicates in INSERT INTO SELECT

If there is already a row with a similar value, SQL may throw an error while using the INSERT INTO SELECT command.

However, we can skip copying duplicate rows using the NOT EXISTS clause. For example,

```
INSERT INTO OldCustomers (customer_id, age)
SELECT customer_id, age
FROM Customers
WHERE NOT EXISTS (
  SELECT customer_id
  FROM OldCustomers
  WHERE OldCustomers.customer_id = Customers.customer_id
);
```

SQL VIEW

A VIEW as we known is a virtual table, through which a selective portion of the data from one or more tables can be seen. Views have the following properties/characteristics:

- o Views do not contain data of their own.
- o Views are used to restrict access to the database or to hide data complexity.
- o A View is stored as a SELECT statement in the database.
- o DML operations on a view such as INSERT, UPDATE, DELETE affects the data in the original table upon which the view is based.
- o A view contains rows and columns, just like a real table.
- o The fields in a View are fields from one or more real tables in the database.
- o A view is a specific representation of data from one or more tables.
- o The tables referred in the views are known as Base tables.
- o Creating a view does not take any storage space as only the query is stored in the data dictionary and the actual data is not stored anywhere.

Uses of Views

There are many reasons for using views in applications such as:

- o Reducing complexity
- o Increasing Security - sensitive information can be excluded from a view
- o Renaming table columns- by giving the different names to columns while creating views.
- o Representing a subset of the data contained in a table.
- o Simplification by joining multiple tables into a single virtual table.
- o Storage Space economy; the database contains only the definition of a view, not a copy of all the data it presents.
- o Giving different users different views created from the same base table.

Creating Views

To create a view, you must meet the following requirements:

- To create a view in your schema, you must have the CREATE VIEW privilege.
- To create a view in another user's schema, you must have the CREATE ANY VIEW system privilege. You can acquire these privileges explicitly or through a role.
- The owner of the view (whether it is you or another user) must have been explicitly granted privileges to access all objects referenced in the view definition.
 - o The owner cannot have obtained these privileges through roles.
- The functionality of the view depends on the privileges of the view owner.
 - o For example, if the owner of the view has only the INSERT privilege for EMP table, then the view can be used only to insert new rows into the EMP table, not to SELECT, UPDATE, or DELETE rows.

Syntax:

```
CREATE [OR REPLACE] VIEW <view_name>
AS
SELECT <column_list>
```

```
FROM <table_name> [WHERE condition]
[WITH CHECK OPTION] [CONSTRAINT constrain_name]
[WITH READ ONLY];
```

In the Syntax, View_name specifies the name of the view and must follow the rules for identifiers. Column_list specifies the name of the columns to be used in view. If the column_list option is not specified, then the view is created with the same columns as specified in the select_statement. Select_statement specifies the SELECT Statement that defines a view.

The view may use the data contained in other views and tables.

WITH CHECK OPTION forces the data modification statements to fulfil the criteria given in the SELECT statement defining the view. It also ensures that the data is visible after the modifications are made permanent.

WITH READ ONLY restrict the DML operations to be performed on the view.

Examples:

Consider the EMPLOYEES table and create a View for the 'EGBE'; cannot access the Salary of all the employees.

```
DESC EMPLOYEES;
CREATE OR REPLACE VIEW EGBE
AS SELECT EID, NAME FROM EMPLOYEES;
```

On Execution the view with Name 'EGBE' is created. Once you create a view it is assumed to be a Table. You can now perform the various DML operations on the View. But make sure if you impose the CHECK OPTIONS, then you cannot perform any operation. Remember that it will also copy the Records contained in the base table.

To Display all the records in the view:

```
SELECT * FROM EGBE;
```

After execution this command will display all the records of EMP table as when view is created all the records are automatically copied into the view. It will display all the records.

Read Only Option

If you use read only option in the view during creation, then you cannot perform any DML operation on the view. If you try to do this then error message will encounter. WITH READ ONLY restrict the DML operations to be performed on the view

Consider the EMPLOYEES table and create a view for the 'EGBEDEEP'; she cannot access the Salary of all the employees. Use the constraint 'WITH READ ONLY'.

```
create view EGBEdeep
as select eid, name from employees with read only;
```

After execution a view with the name 'EGBEdeep' will be created. In the view because the option WITH READ ONLY is used it restricts any DML operations to be executed.

Modifying Views

You can use the OR REPLACE option to modify the view. If the view exists it will be replaced with the new definition or a new view will be created. We can use Create or Replace option to create views instead of dropping the view and recreating it. With this option the privileges granted on the view are preserved, but the dependent stored programs and view become invalid.

The view will become invalid whenever the base table is altered. We can recompile a view using

the Alter view statement, but oracle automatically recompiles the view once it is accessed. On recompiling, the dependent objects become invalid.

Restrictions on DML Statements

There are certain rules that have to be followed to perform the DML operations. They are:

- o You cannot insert the rows into view if the base table has certain columns with NOT NULL constraint that does not appear in the view.
- o If a view is defined by a query that contains SET or DISTINCT operators, a GROUP BY clause, or a group function, then rows cannot be inserted into, updated in, or deleted from the base tables using the view.
- o If a view is defined with WITH CHECK OPTION, a row cannot be inserted into or updated in, the base table (using the view), if the view cannot select the row from the base table.
- o If view contains WITH READ ONLY Constraint, then you cannot perform any DML operations.

Renaming Columns

Views can be used to rename the columns without affecting the base tables provided that the number of columns in a view must match the number of columns specified in the select statement.

Create a view EGBE from the table employees with different column names,

```
CREATE OR REPLACE VIEW EGBE
(ENO, ENAME)
AS SELECT EID, NAME FROM EMPLOYEES;
```

After execution the view EGBE will be created with the different column names

The SELECT Statement

The most commonly used statement in SQL is the SELECT statement. The Statement retrieves data from the database and returns the data to the user. The EMPLOYEE table example illustrates a typical example of a SELECT statement situation.

General Rules of Syntax

Syntax in SQL is quite flexible, although there are rules to follow as in any programming language. A simple query illustrates the basic syntax of an SQL select statement. Pay close attention to the case, spacing, and logical separation of the components of each query by SQL keywords.

```
SELECT NAME, STARTTERM, ..., ENDTERM
FROM EMPLOYEE
WHERE NAME = 'OKORONKWO';
```

In this example everything is capitalized, but it doesn't have to be. The preceding query would work just as well if it were written like this:

```
select name, startterm, ..., endterm
from employee
where name = 'OKORONKWO';
```

Notice that OKORONKWO appears in capital letters in both examples. Although actual SQL statements are not case sensitive, references to data in a database are. For instance, many companies store their data in uppercase. In the preceding example, assume that the column name stores its contents in uppercase. Therefore, a query searching for 'Okoronkwo' in the name column would not find any data to return. Check your implementation and/or company policies for any case requirements.

NOTE: Commands in SQL are not case sensitive.

Take another look at the sample query. Is there something magical in the spacing? Again the answer is no. The following code would work as well:

select name, startterm, ..., endterm from employee where name = ' OKORONKWO';

However, some regard for spacing and capitalization makes your statements much easier to read. It also makes your statements much easier to maintain when they become a part of your project. Another important feature for Oracle DB is; semicolon (;) the sample query is the semicolon at the end of the expression. This punctuation mark tells the command-line SQL program that your query is complete.

If the magic isn't in the capitalization or the format, then just which elements are important? The answer is keywords, or the words in SQL that are reserved as a part of syntax. (Depending on the SQL statement, a keyword can be either a mandatory element of the statement or optional.) The keywords in the example above are:

- SELECT
- FROM
- WHERE

The Building Blocks of Data Retrieval: SELECT and FROM

As your experience with SQL grows, you will notice that you are typing the words SELECT and FROM more than any other words in the SQL vocabulary. They aren't as glamorous as CREATE or as ruthless as DROP, but they are indispensable to any conversation you hope to have with the computer concerning data retrieval. And isn't data retrieval the reason that you entered mountains of information into your very expensive database in the first place?

This discussion starts with SELECT because most of your statements will also start with SELECT:

SYNTAX:

SELECT <COLUMN NAMES>

The SELECT does not work alone. If you typed just SELECT into your system, you might get the following response:

SELECT;

You get an error display like:

SELECT

*

ERROR at line 1: ORA-00936: missing expression

The asterisk under the offending line indicates where Oracle7 thinks the offense occurred. The error message tells you that something is missing. That something is the FROM clause:

SYNTAX:

FROM <TABLE>

Together, the statements SELECT and FROM begin to unlock the power behind your database.

NOTE: keywords clauses. At this point you may be wondering what the difference is between a keyword, a statement, and a clause. SQL **keywords** refer to individual SQL elements, such as SELECT and FROM. A **clause** is a part of an SQL statement; for example, SELECT column1, column2, ... is a clause. You combine SQL clauses to form a complete SQL **statement**. For example, you can combine a SELECT clause and a FROM clause to write an SQL statement.

NOTE: Each implementation of SQL has a unique way of indicating errors. Microsoft Query, for example, says it can't show the query, leaving you to find the problem. Borland's Interbase pops up a dialog box with the error. Personal Oracle7, the engine used in the preceding example, gives you an error number (so you can look up the detailed explanation in your manuals) and a short explanation of the problem.

Terminating an SQL Statement

In some implementations of SQL, the semicolon (;) at the end of the statement tells the interpreter that you are finished writing the query. For example, Oracle's SQL*PLUS won't execute the query until it finds a semicolon (or a slash). On the other hand, some implementations of SQL do not use the semicolon as a terminator. For example, Microsoft Query and Borland's ISQL don't require a terminator, because your query is typed in an edit box and executed when you push a button.

Changing the Order of the Columns

To specify the order of the columns, you could type something like:

SQL> SELECT payee, remarks, amount, check# from checks;

Notice that each column name is listed in the SELECT clause. The order in which the columns are listed is the order in which they will appear in the output. Notice both the commas that separate the column names and the space between the final column name and the subsequent clause (in this case FROM). The output would look like this:

OUTPUT:

PAYEE	REMARKS	AMOUNT	CHECK#
Ma Bell	Have sons next time	150	1
Reading R. R.	Train to Chicago	245.34	2
Ma Bell	Cellular Phone	200.32	3
Local Utilities	Gas	98	4
Joes Stale \$ Dent	Groceries	150	5
Cash	Wild Night Out	25	6
Joans Gas	Gas	25.1	7

7 rows selected.

Another way to write the same statement follows.

**SELECT payee, remarks, amount, check#
FROM checks;**

Notice that the FROM clause has been carried over to the second line. This convention is a matter of personal taste when writing SQL code.

The output is identical because only the format of the statement changed. Now that you have established control over the order of the columns, you will be able to specify which columns you want to see.

Selecting Different Tables

Suppose you had a table called DEPOSITS with this structure:

DEPOSIT#	WHOPAI D	AMOUNT	REMARKS
1	Rich Uncle	200	Take off Xmas list
2	Employer	1000	15 June Payday
3	Credit Union	500	Loan

With a single change you have a new data source.

You would simply change the FROM clause to the desired table and type the following statement:

SQL> select * from deposits

The result is

DEPOSIT#	WHOPAI D	AMOUNT	REMARKS
1	Rich Uncle	200	Take off Xmas list
2	Employer	1000	15 June Payday
3	Credit Union	500	Loan

Queries with Distinction

If you look at the original table, CHECKS, you see that some of the data repeats. For example, if you looked at the AMOUNT column using:

```
SQL> select amount from checks;
```

you would see

```
AMOUNT
```

```
-----
```

```
150
```

```
245.34
```

```
200.32
```

```
98
```

```
150
```

```
25
```

```
25.1
```

Notice that the amount 150 is repeated. What if you wanted to see how many different amounts were in this column? Try this:

```
SQL> select DISTINCT amount from checks;
```

The result would be

```
AMOUNT
```

```
-----
```

```
25
```

```
25.1
```

```
98
```

```
150
```

```
200.32
```

```
245.34
```

6 rows selected.

Notice that only six rows are selected. Because you specified DISTINCT, only one instance of the duplicated data is shown, this means that one less row is returned. ALL is a keyword that is implied in the basic SELECT statement. You almost never see ALL because SELECT <Table> and SELECT ALL <Table> have the same result.

Exercise

1. Do the following statements return the same or different output:

```
SELECT * FROM CHECKS;
```

```
select * from checks;?
```

2. The following queries do not work. Why not?

a. Select *

b. Select * from checks

c. Select amount name payee FROM checks;

3. Which of the following SQL statements will work?

a. select *

from checks;

b. select * from checks;

c. select * from checks

/

4. Using the CHECKS table from earlier today, write a query to return just the check numbers and the remarks.

5. Rewrite the query from exercise 1 so that the remarks will appear as the first column in your query results.

6. Using the CHECKS table, write a query to return all the unique remarks.

Clauses in SQL

Objectives

By the end of the day you will understand and be able to use the following clauses you can use with Select statement:

- WHERE
- STARTING WITH
- ORDER BY
- GROUP BY
- HAVING

The general syntax for a SELECT statement is:

SYNTAX:

```
SELECT [DISTINCT | ALL] { *
                        | { [schema.]{table | view | snapshot}.*
                        | expr } [ [AS] c_alias ]... }
FROM [schema.]{table | view | snapshot}...
[WHERE condition ]
[GROUP BY expr [, expr] ... [HAVING condition] ]
[{UNION | UNION ALL | INTERSECT | MINUS} SELECT command ]
[ORDER BY {expr|position} [ASC | DESC]
  [, {expr|position} [ASC | DESC]] ...]
```

NOTE: The preceding syntax will generally work with any SQL engine, but you may find some slight variations.

Don't worry about the exact syntax-it varies slightly from implementation to implementation anyway. Instead, focus on the relationships. At the top of this statement is SELECT, SELECT is followed by FROM, which should appear with every SELECT statement you typed. WHERE, GROUP BY, HAVING, and ORDER BY all follow. Each clause plays an important part in selecting and manipulating data.

The WHERE Clause

Using just SELECT and FROM, you are limited to returning every row in a table. With WHERE in your vocabulary, you can be more selective. To find all the checks you wrote with a value of more than 100 dollars, write this:

```
SQL> SELECT *
2 FROM CHECKS
3 WHERE AMOUNT > 100;
```

The WHERE clause returns the instances in the table that meet the required condition:

WHERE can also solve other popular puzzles. Given a table of names and locations, you can ask that popular question, Where's Waldo?

```
SQL> SELECT LOCATION AS "WHERE'S WALDO?"
2 FROM PUZZLE
3 WHERE NAME = 'WALDO';
```

OUTPUT:

```
WHERE'S WALDO?
-----
GARAGE
```

This query shows that the column used in the condition of the WHERE statement does not have to be mentioned in the SELECT clause. In this example you selected the location column but used WHERE on the name, which is perfectly legal. Also notice the AS on the SELECT line. AS is an

optional assignment operator, assigning the alias WHERE'S WALDO? to LOCATION. You might never see the AS again, because it involves extra typing. In most implementations of SQL you can type

```
SQL> SELECT LOCATION "WHERE'S WALDO?"
2 FROM PUZZLE
3 WHERE NAME ='WALDO';
```

and get the same result as the previous query without using AS:

OUTPUT:

```
WHERE'S WALDO?
-----
GARAGE
```

After SELECT and FROM, WHERE, is the third most frequently used SQL term.

The STARTING WITH Clause

STARTING WITH is an addition to the WHERE clause that works exactly like LIKE(<exp>%).

Compare the results of the following query:

```
SELECT PAYEE, AMOUNT, REMARKS
FROM CHECKS
WHERE PAYEE LIKE('Ca%');
```

and

```
SELECT PAYEE, AMOUNT, REMARKS
FROM CHECKS
WHERE PAYEE STARTING WITH('Ca');
```

The results are identical. You can even use them together, as shown here:

```
SELECT PAYEE, AMOUNT, REMARKS
FROM CHECKS
WHERE PAYEE STARTING WITH('Ca')
OR
REMARKS LIKE 'G%';
```

WARNING: STARTING WITH is a common feature of many implementations of SQL. Check your implementation before you grow fond of it.

The ORDER BY Clause

SELECT FROM gives you a listing, and unless you have defined a primary key, your query comes out in the order the rows were entered. Consider the following query on CHECKS table:

```
SELECT * FROM CHECKS;
```

The order of the output is exactly the same order as the order in which the data was entered.

The ORDER BY clause gives you a way of ordering your results. For example, to order the preceding listing by check number, you would use the following ORDER BY clause:

```
SQL> SELECT *
2 FROM CHECKS
3 ORDER BY CHECK#;
```

Now the data is ordered the way you want it, not the way in which it was entered. As the following example shows, ORDER requires BY; BY is not optional the following query.

```
SQL> SELECT * FROM CHECKS ORDER CHECK#;
```

Display the following error flag:

```
SELECT * FROM CHECKS ORDER CHECK#
*
ERROR at line 1:
```

ORA-00924: missing BY keyword

If you want to list the data in reverse order, with the highest number or letter first? The following query generates a list of PAYEEs that starts at the end of the alphabet:

```
SQL> SELECT *
2 FROM CHECKS
3 ORDER BY PAYEE DESC;
```

The DESC at the end of the ORDER BY clause orders the list in descending order instead of the default (ascending) order.

This query also shows that the expression used after the ORDER BY clause does not have to be in the SELECT statement. Although you selected only PAYEE and AMOUNT, you were still able to order the list by CHECK#.

You can also use ORDER BY on more than one field. To order CHECKS by PAYEE and REMARKS, you would query as follows:

```
SQL> SELECT *
2 FROM CHECKS
3 ORDER BY PAYEE, REMARKS;
```

Adding the field REMARKS to the ORDER BY clause puts the entries in alphabetical order according to REMARKS. Does the order of multiple columns in the ORDER BY clause make a difference? Try the same query again but reverse PAYEE and REMARKS:

```
SQL> SELECT *
2 FROM CHECKS
3 ORDER BY REMARKS, PAYEE;
```

The results are completely different. Here's how to list one column in alphabetical order and list the second column in reverse alphabetical order:

```
SQL> SELECT *
2 FROM CHECKS
3 ORDER BY PAYEE ASC, REMARKS DESC;
```

In this example PAYEE is sorted alphabetically, and REMARKS appears in descending order.

TIP: If you know that a column you want to order your results by is the first column in a table, then you can type ORDER BY 1 in place of spelling out the column name. See the following example.

```
SQL> SELECT *
2 FROM CHECKS
3 ORDER BY 1;
```

This result is identical to the result produced by the SELECT statement that you used earlier: SELECT * FROM CHECKS ORDER BY CHECK#;

The GROUP BY Clause

The SQL GROUP BY clause is used along with the SQL aggregate functions and specifies the groups where selected rows are placed. When one or more aggregate functions are presented in the SQL SELECT column list, the SQL GROUP BY clause calculates a summary value for each group.

Syntax:

```
SQL> SELECT column_name, aggregate_function (column_name)
FROM table_name
WHERE column_name operator value
GROUP BY column_name;
```

GROUP BY is an invaluable SQL clause for organizing and performing operations on information

about several different items you want to aggregate. For instance, what if you wanted to get information about employee attendance in several different time periods to do comparisons and charting?

Examples:

a. To find out the Total salary of all the Departments.

```
SQL> SELECT DEPTNO, SUM (SAL) "DEPARTEMENT SALARY"
2 FROM EMP GROUP BY DEPTNO;
```

DEPTNO	DEPARTEMENT SALARY
10	8750
20	10875
30	9400

b. To list the average salary of each job in the EMP table

```
SQL) SELECT JOB, AVG (SAL)
FROM EMP GROUP BY JOB;
```

JOB	SAL
ANALYST	3000
CLERK	1037.5
MANAGER	2758.33333
PRESIDENT	5000
SALESMAN	1400

To find out the jobs of individual departments

```
SQL> SELECT JOB, COUNT (JOB) "TOTAL JOBS" FROM EMP
GROUP BY DEPTNO, JOB;
```

TOTAL JOBS	JOB
1	CLERK
1	MANAGER
1	PRESIDENT
2	ANALYST
2	CLERK
1	MANAGER
1	CLERK
1	MANAGER
4	SALESMAN

If you wanted to find the total amount of money spent from the CHECKS table, you would write:

```
SELECT SUM(AMOUNT)
FROM CHECKS;
```

OUTPUT

```
SUM
=====
1159.87
```

This statement returns the sum of the column AMOUNT.

If you wanted to find out how much you have spent on each PAYEE? SQL helps you with the GROUP BY clause to find out whom you have paid and how much, you would query like this:

```
SELECT PAYEE, SUM(AMOUNT)
FROM CHECKS
GROUP BY PAYEE;
```

If you had tried this query with only the FROM CHECKS that follows, here's what you would see:

```
SELECT PAYEE, SUM(AMOUNT)
FROM CHECKS;
```

Dynamic SQL Error

-SQL error code = -104

-invalid column reference

SQL is complaining about the combination of the normal column and the aggregate function. This condition requires the GROUP BY clause. GROUP BY runs the aggregate function described in the SELECT statement for each grouping of the column that follows the GROUP BY clause.

What would happen if you tried to group by more than one column? Try this:

```
SELECT PAYEE, SUM(AMOUNT), COUNT(PAYEE)
FROM CHECKS
GROUP BY PAYEE, REMARKS;
```

The next example finds the largest and smallest amounts, grouped by REMARKS:

```
SELECT MIN(AMOUNT), MAX(AMOUNT)
FROM CHECKS
GROUP BY REMARKS;
```

If you try to include in the select statement a column that has several different values within the group formed by GROUP BY:

```
SELECT PAYEE, MAX(AMOUNT), MIN(AMOUNT)
FROM CHECKS
GROUP BY REMARKS;
```

You will get this:

Dynamic SQL Error

-SQL error code = -104

-invalid column reference

This query tries to group CHECKS by REMARK. When the query finds two records with the same REMARK but different PAYEEs, it throws an error.

The rule is, Don't use the SELECT statement on columns that have multiple values for the GROUP BY column. The reverse is not true. You can use GROUP BY on columns not mentioned in the SELECT statement.

For example:

```
SELECT PAYEE, COUNT(AMOUNT)
FROM CHECKS
GROUP BY PAYEE, AMOUNT;
```

This query shows how many checks you had written for identical amounts to the same PAYEE. Its real purpose is to show that you can use AMOUNT in the GROUP BY clause, even though it is not

mentioned in the SELECT clause.

Try moving AMOUNT out of the GROUP BY clause and into the SELECT clause, like this:

```
SELECT PAYEE, AMOUNT, COUNT(AMOUNT)
FROM CHECKS
GROUP BY PAYEE;
```

You get a Dynamic SQL Error

-SQL error code = -104

-invalid column reference

SQL cannot run the query, which makes sense if you play the part of SQL for a moment. Say you had to group the following lines:

```
SELECT PAYEE, AMOUNT, REMARKS
FROM CHECKS
WHERE PAYEE ='Cash';
```

If the user asked you to output all three columns and group by PAYEE only, where would you put the unique remarks? Remember you have only one row per group when you use GROUP BY. SQL can't do two things at once, so it complains: Error #31: Can't do two things at once.

The HAVING Clause

How can you qualify the data used in your GROUP BY clause? HAVING enables you to use aggregate functions in a comparison statement, providing for aggregate functions what WHERE provides for individual rows.

Use the table ORGCHART and try this:

```
SELECT * FROM ORGCHART;
```

If you wanted to group the output into divisions and show the average salary in each division, you would type:

```
SELECT TEAM, AVG(SALARY)
FROM ORGCHART
GROUP BY TEAM;
```

The following statement qualifies this query to return only those departments with average salaries under 38000:

```
SELECT TEAM, AVG(SALARY)
FROM ORGCHART
WHERE AVG(SALARY) < 38000
GROUP BY TEAM;
```

Dynamic SQL Error

-SQL error code = -104

-Invalid aggregate reference

This error occurred because WHERE does not work with aggregate functions. To make this query work, you need something new: the HAVING clause. If you type the following query, you get what you ask for:

```
SELECT TEAM, AVG(SALARY)
FROM ORGCHART
GROUP BY TEAM
HAVING AVG(SALARY) < 38000;
```

Does HAVING work with nonaggregate expressions? Try this:

```
SELECT TEAM, AVG(SALARY)
```

```
FROM ORGCHART
GROUP BY TEAM
HAVING SALARY < 38000;
```

Why is this result different from the last query?

The HAVING AVG(SALARY) < 38000 clause evaluated each grouping and returned only those with an average salary of under 38000, just what you expected.

HAVING SALARY < 38000, on the other hand, will produce a different outcome.

Take on the role of the SQL engine again. If the user asks you to evaluate and return groups of divisions where SALARY < 38000, you would examine each group and reject those where an individual SALARY is greater than 38000. In each division, you would find at least one salary greater than 38000. Therefore, you would reject all. What you really asked was Select all groups where no individual makes more than 38000. Don't you just hate it when the computer does exactly what you tell it to?

WARNING: Some implementations of SQL return an error if you use anything other than an aggregate function in a HAVING clause. Don't bet on using the previous example until you check the implementation of the particular SQL you use.

Can you use more than one condition in your HAVING clause? Try this:

```
SELECT TEAM, AVG(SICKLEAVE),AVG(ANNUALLEAVE)
FROM ORGCHART
GROUP BY TEAM
HAVING AVG(SICKLEAVE)>25 AND
AVG(ANNUALLEAVE)<20;
```

The following table is grouped by TEAM. It shows all the teams with SICKLEAVE averages above 25 days and ANNUALLEAVE averages below 20 days.

You can also use an aggregate function in the HAVING clause that was not in the SELECT statement. For example:

```
SELECT TEAM, AVG(SICKLEAVE),AVG(ANNUALLEAVE)
FROM ORGCHART
GROUP BY TEAM
HAVING COUNT(Team) > 1;
```

This query returns the number of TEAMS with more than one member. COUNT(Team) is not used in the SELECT statement but still functions as expected in the HAVING clause.

The other logical operators all work well within the HAVING clause. Consider this:

```
SELECT TEAM,MIN(SALARY),MAX(SALARY)
FROM ORGCHART
GROUP BY TEAM
HAVING AVG(SALARY) > 37000
OR
MIN(SALARY) > 32000;
```

The operator IN also works in a HAVING clause, as demonstrated here:

```
SELECT TEAM,AVG(SALARY)
FROM ORGCHART
GROUP BY TEAM
HAVING TEAM IN ('CS','ST');
```

Combining Clauses

Nothing exists in a vacuum, so this section takes you through some composite examples that demonstrate how combinations of clauses perform together.

Example

Find all the checks written for Cash and Gas in the CHECKS table and order them by REMARKS.

```
SELECT PAYEE, REMARKS
FROM CHECKS
WHERE PAYEE = 'Cash'
OR REMARKS LIKE 'Ga%'
ORDER BY REMARKS;
```

Note the use of LIKE to find the REMARKS that started with Ga. With the use of OR, data was returned if the WHERE clause met either one of the two conditions.

What if you asked for the same information and group it by PAYEE? The query would look something like this:

```
SELECT PAYEE, REMARKS
FROM CHECKS
WHERE PAYEE = 'Cash'
OR REMARKS LIKE 'Ga%'
GROUP BY PAYEE
ORDER BY REMARKS;
```

This query would not work because the SQL engine would not know what to do with the remarks. Remember that whatever columns you put in the SELECT clause must also be in the GROUP BY clause--unless you don't specify any columns in the SELECT clause.

Example 2

Using the table ORGCHART, find the salary of everyone with less than 25 days of sick leave. Order the results by NAME.

```
SELECT NAME, SALARY
FROM ORGCHART
WHERE SICKLEAVE < 25
ORDER BY NAME;
```

This query is straightforward and enables you to use your new-found skills with WHERE and ORDER BY.

Example 3

Again, using ORGCHART, display TEAM, AVG(SALARY), AVG(SICKLEAVE), and AVG(ANNUALLEAVE) on each team:

```
SELECT TEAM,
AVG(SALARY),
AVG(SICKLEAVE),
AVG(ANNUALLEAVE)
FROM ORGCHART
GROUP BY TEAM;
```

An interesting variation on this query follows. See if you can figure out what happened:

```
SELECT TEAM,
AVG(SALARY),
AVG(SICKLEAVE),
AVG(ANNUALLEAVE)
FROM ORGCHART
```

```
GROUP BY TEAM
ORDER BY NAME;
```

A simpler query using ORDER BY might offer a clue:

```
SELECT NAME, TEAM
FROM ORGCHART
ORDER BY NAME, TEAM;
```

NAME	TEAM
=====	=====
ADAMS	ST
FURY BM	
MERRICK	ST
MEZA BM	
PRECOURT	CS
RICHARDSON	MKT
STOKES	MKT
WILKES	MKT

When the SQL engine got around to ordering the results of the query, it used the NAME column (remember, it is perfectly legal to use a column not specified in the SELECT statement), ignored duplicate TEAM entries, and came up with the order ST, BM, CS, and MKT. Including TEAM in the ORDER BY clause is unnecessary, because you have unique values in the NAME column. You can get the same result by typing this statement:

```
SELECT NAME, TEAM
FROM ORGCHART
ORDER BY NAME;
```

While you are looking at variations, don't forget you can also reverse the order:

```
SELECT NAME, TEAM
FROM ORGCHART
ORDER BY NAME DESC;
```

Example 4:

Is it possible to use everything you have learned in one query? It is, but the results will be convoluted because in many ways you are working with apples and oranges or aggregates and non aggregates. For example, WHERE and ORDER BY are usually found in queries that act on single rows, such as this:

```
SELECT *
FROM ORGCHART
ORDER BY NAME DESC;
```

GROUP BY and HAVING are normally seen in the company of aggregates:

```
SELECT PAYEE,
SUM(AMOUNT) TOTAL,
COUNT(PAYEE) NUMBER_WRITTEN
FROM CHECKS
GROUP BY PAYEE
HAVING SUM(AMOUNT) > 50;
```

You have seen that combining these two groups of clauses can have unexpected results, including the following:

```

SELECT PAYEE,
SUM(AMOUNT) TOTAL,
COUNT(PAYEE) NUMBER_WRITTEN
FROM CHECKS
WHERE AMOUNT >= 100
GROUP BY PAYEE
HAVING SUM(AMOUNT) > 50;

```

Compare these two result sets and examine the raw data:

```

SELECT PAYEE, AMOUNT
FROM CHECKS
ORDER BY PAYEE;

```

You see how the WHERE clause filtered out all the checks less than 100 dollars before the GROUP BY was performed on the query. We are not trying to tell you not to mix these groups—you may have a requirement that this sort of construction will meet. However, you should not casually mix aggregate and non-aggregate functions. The previous examples have been tables with only a handful of rows. (Otherwise, you would need a cart to carry this book.) In the real world you will be working with thousands and thousands (or billions and billions) of rows, and the subtle changes caused by mixing these clauses might not be so apparent.

Exercises

1. Which clause works just like LIKE(<exp>%)?
2. What is the function of the GROUP BY clause, and what other clause does it act like?
3. Will this SELECT work?

```

SQL> SELECT NAME, AVG(SALARY), DEPARTMENT
      FROM PAY_TBL
      WHERE DEPARTMENT = 'ACCOUNTING'
      ORDER BY NAME
      GROUP BY DEPARTMENT, SALARY;

```
4. When using the HAVING clause, do you always have to use a GROUP BY also?
5. Can you use ORDER BY on a column that is not one of the columns in the SELECT statement?
6. Using the ORGCHART table from the preceding examples, find out how many people on each team have 30 or more days of sick leave.
7. Using the CHECKS table, write a SELECT that will return the following:

Expressions, Conditions, and Operators

Objectives

In the previous section on “The SELECT Statement,” you used SELECT and FROM to manipulate data in interesting (and useful) ways. Here you learn more about SELECT and FROM and expand the basic query with some new terms to go with query, table, and row, as well as a new clause and a group of handy items called operators.

- Know what an expression is and how to use it
- Know what a condition is and how to use it
- Be familiar with the basic uses of the WHERE clause
- Be able to use arithmetic, comparison, character, logical, and set operators
- Have a working knowledge of some miscellaneous operators

NOTE: We used Oracle's Personal Oracle7 to generate the examples. Other implementations of SQL may differ slightly in the way in which commands are entered or output is displayed, but the

results are basically the same for all implementations that conform to the ANSI standard.

Expressions

An *expression* returns a value. Expression types are very broad, covering different data types such as String, Numeric, and Boolean. In fact, pretty much anything following a clause (SELECT or FROM, for example) is an expression. In the following example amount is an expression that returns the value contained in the amount column.

SELECT amount FROM checks;

In the following statement NAME, ADDRESS, PHONE and ADDRESSBOOK are expressions:

```
SELECT NAME, ADDRESS, PHONE
FROM ADDRESSBOOK;
```

Now, examine the following expression:

```
WHERE NAME = 'BROWN'
```

It contains a condition, NAME = 'BROWN', which is an example of a Boolean expression. NAME = 'BROWN' will be either TRUE or FALSE, depending on the condition.

Conditions

If you ever want to find a particular item or group of items in your database, you need one or more conditions. Conditions are contained in the WHERE clause. In the preceding example, the condition is: NAME = 'BROWN'

To find everyone in your organization who worked more than 100 hours last month, your condition would be: NUMBEROFHOURS > 100

Conditions enable you to make selective queries. In their most common form, conditions comprise a variable, a constant, and a comparison operator. In the first example the variable is NAME, the constant is 'BROWN', and the comparison operator is =. In the second example the variable is NUMBEROFHOURS, the constant is 100, and the comparison operator is >. You need to know about two more elements before you can write conditional queries: the WHERE clause and operators.

The WHERE Clause

The syntax of the WHERE clause is

```
WHERE <SEARCH CONDITION>
```

The WHERE, simply causes your queries to be more selective. Without the WHERE clause, the most useful thing you could do with a query is display all records in the selected table(s). For example:

```
SQL> SELECT * FROM BIKES;
```

lists all rows of data in the table BIKES.

NAME	FRAMESIZE	COMPOSITION	MILESRIIDEN TYPE
TREK 2300	22.5	CARBON FIBER	3500 RACING
BURLEY	22	STEEL	2000 TANDEM
GIANT	19	STEEL	1500 COMMUTER
FUJI	20	STEEL	500 TOURING
SPECIALIZED	16	STEEL	100 MOUNTAIN
CANNONDALE	22.5	ALUMINUM	3000 RACING

6 rows selected.

If you wanted a particular bike, you could type

```
SQL> SELECT *
FROM BIKES
WHERE NAME = 'BURLEY';
```

which would yield only one record:

NAME	FRAMESIZE	COMPOSITION	MILES RIDDEN TYPE
BURLEY	22	STEEL	2000 TANDEM

This simple example shows how you can place a condition on the data that you want to retrieve.

Operators

Operators are the elements you use inside an expression to articulate how you want specified conditions to retrieve data. Operators fall into six groups: arithmetic, comparison, character, logical, set, and miscellaneous.

Arithmetic Operators

The arithmetic operators are plus (+), minus (-), divide (/), multiply (*), and modulo (%). The first four are self-explanatory. Modulo returns the integer remainder of a division. Here are two examples:

$5 \% 2 = 1$

$6 \% 2 = 0$

The modulo operator does not work with data types that have decimals, such as Real or Number. If you place several of these arithmetic operators in an expression without any parentheses, the operators are resolved in this order: multiplication, division, modulo, addition, and subtraction. For example, the expression

$2 * 6 + 9 / 3$ equals $12 + 3 = 15$

However, the expression

$2 * (6 + 9) / 3$ equals $2 * 15 / 3 = 10$

Watch where you put those parentheses! Sometimes the expression does exactly what you tell it to do, rather than what you want it to do.

The following sections examine the arithmetic operators in some detail and give you a chance to write some queries.

Plus (+)

You can use the plus sign in several ways. Type the following statement to display the PRICE table:

SQL> SELECT * FROM PRICE;

ITEM	WHOLESALE
TOMATOES	.34
POTATOES	.51
BANANAS	.67

Now type:

SQL> SELECT ITEM, WHOLESALE, WHOLESALE + 0.15
FROM PRICE;

Here the + adds 15 cents to each price to produce the following:

ITEM	WHOLESALE	WHOLESALE+0.15
TOMATOES	.34	.49
POTATOES	.51	.66
BANANAS	.67	.82

What is this last column with the unattractive column heading WHOLESALE+0.15? It's not in the original table. (Remember, you used * in the SELECT clause, which causes all the columns to be shown.) SQL allows you to create a virtual or derived column by combining or modifying existing columns.

WHOLESALE+0.15 is not a permanent part of it. In fact, the column heading is so unattractive that you should do something about it.

Type the following:

SELECT ITEM, WHOLESALE, (WHOLESALE + 0.15) RETAIL

FROM PRICE;

Here's the result:

ITEM	WHOLESALE	RETAIL
TOMATOES	.34	.49
POTATOES	.51	.66
BANANAS	.67	.82

This is wonderful! Not only can you create new columns, but you can also rename them on the fly. You can rename any of the columns using the syntax `column_name alias` (note the space between `column_name` and `alias`).

NOTE: Some implementations of SQL use the syntax `<column name = alias>`. The preceding example would be written as follows:

```
SELECT ITEM = PRODUCE,
       WHOLESALE,
       WHOLESALE + 0.25 = RETAIL,
FROM PRICE;
```

Check your implementation for the exact syntax.

You might be wondering what use aliasing is if you are not using command-line SQL. Fair enough. Have you ever wondered how report builders work? Someday, when you are asked to write a report generator, you'll remember this and not spend weeks reinventing what Dr. Codd and IBM have wrought.

In some implementations of SQL, the plus sign does double duty as a character operator. You'll see that side later.

Minus (-)

Minus also has two uses. First, it can change the sign of a number. You can use the table HILOW to demonstrate this function.

```
SELECT * FROM HILOW;
STATE  HIGHTEMP  LOWTEMP
CA      -50      120
FL       20     110
LA       15      99
ND      -70     101
NE      -60     100
```

For example, here's a way to manipulate the data:

```
SELECT STATE, -HIGHTEMP LOWS, -LOWTEMP HIGHS
FROM HILOW;
STATE  LOWS  HIGHS
CA      50  -120
FL     -20  -110
LA     -15   -99
ND      70  -101
NE      60  -100
```

The second (and obvious) use of the minus sign is to subtract one column from another. For example:

```
SELECT STATE,
       2 HIGHTEMP LOWS,
       3 LOWTEMP HIGHS,
       4 (LOWTEMP - HIGHTEMP) DIFFERENCE
       5 FROM HILOW;
```

```
STATE  LOWS  HIGHS  DIFFERENCE
```


CA	-50	120	170
FL	20	110	90
LA	15	99	84
ND	-70	101	171
NE	-60	100	160

Notice the use of aliases to fix the data that was entered incorrectly. This remedy is merely a temporary patch, though, and not a permanent fix. You should see to it that the data is corrected and entered correctly in the future.

This query not only fixed (at least visually) the incorrect data but also created a new column containing the difference between the highs and lows of each state.

If you accidentally use the minus sign on a character field, you get something like this:

```
SELECT -STATE FROM HILOW;
```

ERROR:

ORA-01722: invalid number
no rows selected

The exact error message varies with implementation, but the result is the same.

Divide (/)

The division operator has only one obvious meaning. Using the table PRICE, type the following:

```
SELECT * FROM PRICE;
ITEM      WHOLESale
-----
TOMATOES      .34
POTATOES      .51
BANANAS      .67
```

You can show the effects of a two-for-one sale by typing the next statement:

```
SELECT ITEM, WHOLESale, (WHOLESale/2) SALEPRICE
FROM PRICE;
```

```
ITEM      WHOLESale SALEPRICE
-----
TOMATOES      .34      .17
POTATOES      .51      .255
BANANAS      .67      .335
```

The use of division in the preceding SELECT statement is straightforward

Multiply (*)

The multiplication operator is also straightforward. Again, using the PRICE table, type the following:

The query below changes the table to reflect an across-the-board 10 percent discount:

```
SELECT ITEM, WHOLESale, WHOLESale * 0.9 NEWPRICE
FROM PRICE;
```

```
ITEM      WHOLESale NEWPRICE
-----
TOMATOES      .34      .306
POTATOES      .51      .459
BANANAS      .67      .603
```

These operators enable you to perform powerful calculations in a SELECT statement.

Modulo (%)

The modulo operator returns the integer remainder of the division operation. Using the table REMAINS, type the following:

```
SELECT * FROM REMAINS;
```

NUMERATOR DENOMINATOR

10	5
8	3
23	9
40	17

You can also create a new column, REMAINDER, to hold the values of NUMERATOR % DENOMINATOR:

```
SELECT NUMERATOR, DENOMINATOR, NUMERATOR%DENOMINATOR REMAINDER
FROM REMAINS;
```

NUMERATOR	DENOMINATOR	REMAINDER
10	5	0
8	3	2
23	9	5

Some implementations of SQL implement modulo as a function called MOD

The following statement produces results that are identical to the results in the preceding statement:

```
SELECT NUMERATOR,
       DENOMINATOR,
       MOD(NUMERATOR,DENOMINATOR) REMAINDER
FROM REMAINS;
```

Precedence

Here we examine the use of precedence in a SELECT statement. Using the database PRECEDENCE, type the following:

```
SELECT * FROM PRECEDENCE;
```

N1	N2	N3	N4
1	2	3	4
13	24	35	46
9	3	23	5

Use the following code segment to test precedence:

```
SELECT
  2 N1+N2*N3/N4,
  3 (N1+N2)*N3/N4,
  4 N1+(N2*N3)/N4
  5 FROM PRECEDENCE;
```

N1+N2*N3/N4	(N1+N2)*N3/N4	N1+(N2*N3)/N4
2.5	2.25	2.5
31.26087	28.152174	31.26087
22.8	55.2	22.8
93	975	93

Notice that the first and last columns are identical. If you added a fourth column $N1+N2* (N3/N4)$, its values would also be identical to those of the current first and last columns.

Comparison Operators

Comparison operators compare expressions and return one of three values: TRUE, FALSE, or Unknown. Wait a minute! Unknown? TRUE and FALSE are self-explanatory, but what is Unknown? To understand how you could get an Unknown, you need to know a little about the concept of

NULL. In database terms NULL is the absence of data in a field. It does not mean a column has a zero or a blank in it. A zero or a blank is a value. NULL means nothing is in that field. If you make a comparison like Field = 9 and the only value for Field is NULL, the comparison will come back Unknown. Because Unknown is an uncomfortable condition, most flavors of SQL change Unknown to FALSE and provide a special operator, IS NULL, to test for a NULL condition.

Here's an example of NULL: Suppose an entry in the PRICE table does not contain a value for WHOLESale. The results of a query might look like this:

```
SQL> SELECT * FROM PRICE;
```

```
ITEM      WHOLESale
```

```
-----
```

```
TOMATOES   .34
```

```
APPLES     .23
```

```
ORANGES
```

Notice that nothing is printed out in the WHOLESale field position for oranges. The value for the field WHOLESale for oranges is NULL. The NULL is noticeable in this case because it is in a numeric column. However, if the NULL appeared in the ITEM column, it would be impossible to tell the difference between NULL and a blank.

Try to find the NULL:

```
SELECT *
```

```
2 FROM PRICE
```

```
3 WHERE WHOLESale IS NULL;
```

```
ITEM      WHOLESale
```

```
ORANGES
```

As you can see by the output, ORANGES is the only item whose value for WHOLESale is NULL or does not contain a value. What if you use the equal sign (=) instead?

```
SELECT *
```

```
FROM PRICE
```

```
WHERE WHOLESale = NULL;
```

```
no rows selected
```

You didn't find anything because the comparison WHOLESale = NULL returned a FALSE--the result was unknown. It would be more appropriate to use an IS NULL instead of =, changing the WHERE statement to WHERE WHOLESale IS NULL. In this case you would get all the rows where a NULL existed.

This example also illustrates both the use of the most common comparison operator, the equal sign (=), and the playground of all comparison operators, the WHERE clause. You already know about the WHERE clause, so here's a brief look at the equal sign.

Equal (=)

Earlier you saw how some implementations of SQL use the equal sign in the SELECT clause to assign an alias. In the WHERE clause, the equal sign is the most commonly used comparison operator. Used alone, the equal sign is a very convenient way of selecting one value out of many. Try this:

```
SQL> SELECT * FROM FRIENDS;
```

LASTNAME	FIRSTNAME	AREACODE	PHONE	ST	ZIP
BUNDY	AL	100	555-1111	IL	22333
MEZA	AL	200	555-2222	UK	
MERRICK	BUD	300	555-6666	CO	80212
MAST	JD	381	555-6767	LA	23456
BULHER	FERRIS	345	555-3223	IL	23332

Let's find JD's row. (On a short list this task appears trivial, but you may have more friends than we do—or you may have a list with thousands of records.)

```
SELECT *
FROM FRIENDS
WHERE FIRSTNAME = 'JD';
```

LASTNAME	FIRSTNAME	AREACODE	PHONE	ST	ZIP
MAST	JD	381		555-6767 LA	23456

We got the result that we expected.

Try this:

```
SELECT * FROM FRIENDS
WHERE FIRSTNAME = 'AL';
```

LASTNAME	FIRSTNAME	AREACODE	PHONE	ST	ZIP
BUNDY	AL		100	555-1111 IL	22333
MEZA	AL	200	555-2222	UK	

NOTE: Here you see that = can pull in multiple records. Notice that ZIP is blank on the second record. ZIP is a character field, and in this particular record the NULL demonstrates that a NULL in a character field is impossible to differentiate from a blank field.

Here's another very important lesson concerning case sensitivity:

```
SELECT * FROM FRIENDS
WHERE FIRSTNAME = 'BUD';
FIRSTNAME
BUD
```

Now try this:

```
select * from friends
where firstname = 'Bud';
```

no rows selected.

Even though SQL syntax is not case sensitive, data is. Most companies prefer to store data in uppercase to provide data consistency. You should always store data either in all uppercase or in all lowercase. Mixing case creates difficulties when you try to retrieve accurate data.

Greater Than (>) and Greater Than or Equal To (>=)

The greater than operator (>) works like this:

```
SELECT *
FROM FRIENDS
WHERE AREACODE > 300;
```

LASTNAME	FIRSTNAME	AREACODE	PHONE	ST	ZIP
MAST	JD	381		555-6767 LA	23456
BULHER	FERRIS	345		555-3223 IL	23332

This example list all the area codes greater than (but not including) 300. To include 300, type this:

```
SELECT * FROM FRIENDS
WHERE AREACODE >= 300;
```

LASTNAME	FIRSTNAME	AREACODE	PHONE	ST	ZIP
BUNDY	AL	100		555-1111 IL	22333
MAST	JD	381		555-6767 LA	23456
BULHER	FERRIS	345		555-3223 IL	23332

With this change you get area codes starting at 300 and going up. You could achieve the same results with the statement AREACODE > 299.

NOTE: Notice that no quotes surround 300 in this SQL statement. Number-defined fields do not require quotes.

Less Than (<) and Less Than or Equal To (<=)

These comparison operators work the same way as > and >= work, only in reverse:

```
SELECT *
  2 FROM FRIENDS
  3 WHERE STATE < 'LA';
```

LASTNAME	FIRSTNAME	AREACODE	PHONE	ST	ZIP
BUNDY	AL	100		555-1111 IL	22333
MERRICK	BUD	300		555-6666 CO	80212
BULHER	FERRIS	345		555-3223 IL	23332

Wait a minute. Did you just use < on a character field? Of course you did. You can use any of these operators on any data type. The result varies by data type. For example, use lowercase in the following state search:

```
SELECT *
  2 FROM FRIENDS
  3 WHERE STATE < 'la';
```

LASTNAME	FIRSTNAME	AREACODE	PHONE	ST	ZIP
BUNDY	AL	100		555-1111 IL	22333
MEZA	AL	200		555-2222 UK	
MERRICK	BUD	300		555-6666 CO	80212
MAST	JD	381		555-6767 LA	23456
BULHER	FERRIS	345		555-3223 IL	23332

Notice that Uppercase is usually sorted before lowercase; therefore, the uppercase codes returned are less than 'la'. Again, to be safe, check your implementation.

To include the state of Louisiana in the original search, type

```
SELECT *
  2 FROM FRIENDS
  3 WHERE STATE <= 'LA';
```

LASTNAME	FIRSTNAME	AREACODE	PHONE	ST	ZIP
BUNDY	AL	100		555-1111 IL	22333
MERRICK	BUD	300		555-6666 CO	80212
MAST	JD	381		555-6767 LA	23456
BULHER	FERRIS	345		555-3223 IL	23332

Inequalities (< > or !=)

When you need to find everything except for certain data, use the inequality symbol, which can be either < > or !=, depending on your SQL implementation. For example, to find everyone who is not AL, type this:

```
SELECT * FROM FRIENDS
  WHERE FIRSTNAME <> 'AL';
```

To find everyone not living in California, type this:

```
SELECT * FROM FRIENDS
  WHERE STATE != 'CA';
```

You can use both symbols, <> and !=, can express "not equals."

Character Operators

You can use character operators to manipulate the way character strings are represented, both in the output of data and in the process of placing conditions on data to be retrieved. This section describes two character operators: the LIKE operator and the || operator, which conveys the concept of character concatenation.

The LIKE

What if you wanted to select parts of a database that fit a pattern but weren't quite exact matches? You could use the equal sign and run through all the possible cases, but that process would be boring and time-consuming. Instead, you could use LIKE. Consider the following:

SQL> **SELECT * FROM PARTS;**

NAME	LOCATION	PARTNUMBER
APPENDIX	MID-STOMACH	1
ADAMS APPLE	THROAT	2
HEART	CHEST	3
SPINE	BACK	4
ANVIL	EAR	5
KIDNEY	MID-BACK	6

How can you find all the parts located in the back? Try this:

SQL> **SELECT * FROM PARTS**
WHERE LOCATION LIKE '%BACK%';

You can see the use of the percent sign (%) in the statement after LIKE. When used inside a LIKE expression, % is a wildcard. What you asked for was any occurrence of BACK in the column location. If you queried

SQL> **SELECT ***
FROM PARTS
WHERE LOCATION LIKE 'BACK%';

you would get any occurrence that started with BACK:

If you type

SQL> **SELECT ***
FROM PARTS
WHERE NAME LIKE 'A%';

you would get any name that starts with A.

Is LIKE case sensitive?

The answer is yes. References to data are always case sensitive.

What if you want to find data that matches all but one character in a certain pattern? In this case you could use a different type of wildcard: the underscore.

Underscore (_)

The underscore is the single-character wildcard. Using a modified version of the table FRIENDS, type this:

To find all the records where STATE starts with C, type the following:

SQL> **SELECT ***
2 FROM FRIENDS
3 WHERE STATE LIKE 'C_';

LASTNAME	FIRSTNAME	AREACODE	PHONE	ST	ZIP
MERRICK	BUD	300	555-6666	CO	80212
PERKINS	ALTON	911	555-3116	CA	95633
BOSS	SIR	204	555-2345	CT	95633

You can use several underscores in a statement:

```
SQL> SELECT *
2 FROM FRIENDS
3 WHERE PHONE LIKE '555-6_6_';
```

LASTNAME	FIRSTNAME	AREACODE	PHONE	ST	ZIP
MERRICK	BUD	300	555-6666	CO	80212
MAST	JD	381	555-6767	LA	23456

The previous statement could also be written as follows:

```
SQL> SELECT *
2 FROM FRIENDS
3 WHERE PHONE LIKE '555-6%';
```

The two wildcards can be combined to find all records with L as the second character:

```
SQL> SELECT *
2 FROM FRIENDS
3 WHERE FIRSTNAME LIKE '_L%';
```

LASTNAME	FIRSTNAME	AREACODE	PHONE	ST	ZIP
BUNDY	AL	100	555-1111	IL	22333
MEZA	AL	200	555-2222	UK	
PERKINS	ALTON	911	555-3116	CA	95633

Concatenation (||)

The || (double pipe) symbol concatenates two strings.

Try this:

```
SQL> SELECT FIRSTNAME || LASTNAME ENTIRE-NAME
FROM FRIENDS;
```

<u>ENTIRE-NAME</u>	
AL	BUNDY
AL	MEZA
BUD	MERRICK
JD	MAST
FERRIS	BULHER
ALTON	PERKINS
SIR	BOSS

7 rows selected.

Notice that || is used instead of +. If you use + to concatenate the strings, the SQL interpreter used for this example (Personal Oracle7) returns the following error:

```
SQL> SELECT FIRSTNAME + LASTNAME ENTIRENAME
FROM FRIENDS;
```

ERROR:

ORA-01722: invalid number

It is looking for two numbers to add and throws the error invalid number when it doesn't find any.

NOTE: Some implementations of SQL use the plus sign to concatenate strings. Check your implementation.

Here's a more practical example using concatenation:

```
SELECT LASTNAME || ', ' || FIRSTNAME NAME
FROM FRIENDS;
NAME
BUNDY , AL
```

MEZA ,AL
 MERRICK ,BUD
 MAST ,JD
 BULHER ,FERRIS
 PERKINS,ALTON
 BOSS ,SIR

7 rows selected.

This statement inserted a comma between the last name and the first name.

NOTE: Notice the extra spaces between the first name and the last name in these examples. These spaces are actually part of the data. With certain data types, spaces are right-padded to values less than the total length allocated for a field. "

So far you have performed the comparisons one at a time. That method is fine for some problems, but what if you need to find all the people at work with last names starting with P who have less than three days of Holiday time?

Logical Operators

logical operators separate two or more conditions in the WHERE clause of an SQL statement. Holiday time is always a hot topic around the workplace. Say you designed a table below called HOLIDAY for the accounting department:

SQL> SELECT * FROM HOLIDAY;

OUTPUT:

LASTNAME	EMPLOYEEENUM	YEARS	LEAVETAKEN
ABLE	101	2	4
BAKER	104	5	23
BLEDSE	107	8	45
BOLIVAR	233	4	80
BOLD	210	15	100
COSTALES	211	10	78

6 rows selected.

Suppose your company gives each employee 12 days of leave each year. Using what you have learned and a logical operator, to find all the employees whose names start with B and who have more than 50 days of leave coming.

```
SQL> SELECT LASTNAME,
2 YEARS * 12 - LEAVETAKEN REMAINING
3 FROM HOLIDAY
4 WHERE LASTNAME LIKE 'B%'
5 AND
6 YEARS * 12 - LEAVETAKEN > 50;
```

The SELECT clause (lines 1 and 2) uses arithmetic operators to determine how many days of leave each employee has remaining. The normal precedence computes YEARS * 12 - LEAVETAKEN. (A clearer approach would be to write (YEARS * 12) - LEAVETAKEN.)

LIKE is used in line 4 with the wildcard % to find all the B names. Line 6 uses the > to find all occurrences greater than 50.

The new element is on line 5. You used the logical operator AND to ensure that you found records that met the criteria in lines 4 and 6.

AND

AND means that the expressions on both sides must be true to return TRUE. If either expression is false, AND returns FALSE. For example, to find out which employees have been with the company for 5 years or less and have taken more than 20 days leave, try this:


```
SQL> SELECT LASTNAME
2 FROM HOLIDAY
3 WHERE YEARS <= 5
4 AND
5 LEAVETAKEN > 20 ;
```

If you want to know which employees have been with the company for 5 years or more and have taken less than 50 percent of their leave, you could write:

```
SQL> SELECT LASTNAME WORKAHOLICS
2 FROM HOLIDAY
3 WHERE YEARS >= 5
4 AND
5 ((YEARS *12)-LEAVETAKEN)/(YEARS * 12) < 0.50;
```

Check these people for burnout. Also check out how we used the AND to combine these two conditions.

OR

You can also use OR to sum up a series of conditions. If any of the comparisons is true, OR returns TRUE. To illustrate the difference, conditions run the last query with OR instead of with AND:

```
SQL> SELECT LASTNAME WORKAHOLICS
2 FROM HOLIDAY
3 WHERE YEARS >= 5
4 OR
5 ((YEARS *12)-LEAVETAKEN)/(YEARS * 12) >= 0.50;
```

The original names are still in the list, but you have three new entries (who would probably resent being called workaholics). These three new names made the list because they satisfied one of the conditions. OR requires that only one of the conditions be true in order for data to be returned.

NOT

NOT means just that if the condition it applies to evaluates to TRUE, NOT make it FALSE. If the condition after the NOT is FALSE, it becomes TRUE. For example, the following SELECT returns the only two names not beginning with B in the table:

```
SQL> SELECT *
2 FROM HOLIDAY
3 WHERE LASTNAME NOT LIKE 'B%';
```

NOT can also be used with the operator IS when applied to NULL. Recall the PRICES table where we put a NULL value in the WHOLESale column opposite the item ORANGES.

To find the non-NULL items, type this:

```
SQL> SELECT *
2 FROM PRICE
3 WHERE WHOLESale IS NOT NULL;
```

Set Operators

On Day 1, "Introduction to SQL," you learned that SQL is based on the theory of sets. The following sections examine set operators.

UNION and UNION ALL

UNION returns the results of two queries minus the duplicate rows. The following two tables represent the rosters of teams:

SQL> SELECT * FROM FOOTBALL; <u>NAME</u>	SQL> SELECT * FROM SOFTBALL; <u>NAME</u>
---	---

ABLE	ABLE
BRAVO	BAKER
CHARLIE	CHARLIE
DECON	DEAN
EXITOR	EXITOR
FUBAR	FALCONER
GOOBER	GOOBER
7 rows selected.	

How many different people play on one team or another?

```
SQL> SELECT NAME FROM SOFTBALL
```

```
2 UNION
```

```
3 SELECT NAME FROM FOOTBALL;
```

```
NAME
```

```
ABLE
```

```
BAKER
```

```
BRAVO
```

```
CHARLIE
```

```
DEAN
```

```
DECON
```

```
EXITOR
```

```
FALCONER
```

```
FUBAR
```

```
GOOBER
```

```
10 rows selected.
```

UNION returns 10 distinct names from the two lists. How many names are on both lists (including duplicates)?

The combined list--courtesy of the UNION ALL statement--has 14 names. UNION ALL works just like UNION except it does not eliminate duplicates. Now to show a list of players who are on both teams. You can't do that with UNION--you need to learn INTERSECT.

INTERSECT

INTERSECT returns only the rows found by both queries. The next SELECT statement shows the list of players who play on both teams:

```
SQL> SELECT * FROM FOOTBALL
```

```
2 INTERSECT
```

```
3 SELECT * FROM SOFTBALL;
```

In this example INTERSECT finds the short list of players who are on both teams by combining the results of the two SELECT statements.

MINUS (Difference)

Minus returns the rows from the first query that were not present in the second. For example:

```
SQL> SELECT * FROM FOOTBALL
```

```
2 MINUS
```

```
3 SELECT * FROM SOFTBALL;
```

The preceding query shows the three football players who are not on the softball team. If you reverse the order, you get the three softball players who aren't on the football team:

```
SQL> SELECT * FROM SOFTBALL
```

```
2 MINUS
```

```
3 SELECT * FROM FOOTBALL;
```

```
NAME
```

BAKER
DEAN
FALCONER

Miscellaneous Operators: IN and BETWEEN

The two operators IN and BETWEEN provide a shorthand for functions you already know how to do. If you wanted to find friends in Colorado, California, and Louisiana, you could type the following:

```
SQL> SELECT *
  2 FROM FRIENDS
  3 WHERE STATE IN('CA','CO','LA');
```

The second example is shorter and more readable than the first. You never know when you might have to go back and work on something you wrote months ago. IN also works with numbers.

If you needed a range of things from the PRICE table, you could write the following:

```
SQL> SELECT *
  2 FROM PRICE
  3 WHERE WHOLESALE > 0.25
  4 AND
  5 WHOLESALE < 0.75;
```

Using BETWEEN, you would write this:

```
SQL> SELECT *
  2 FROM PRICE
  3 WHERE WHOLESALE BETWEEN 0.25 AND 0.75;
```

to get the same result

ITEM	WHOLESALE
TOMATOES	.34
POTATOES	.51
BANANAS	.67
TURNIPS	.45

Again, the second example is a cleaner, more readable solution than the first.

NOTE: If a WHOLESALE value of 0.25 existed in the PRICE table, that record would have been retrieved also. Parameters used in the BETWEEN operator are inclusive parameters.

SQL EXISTS

The EXISTS condition is considered "to be met" if the subquery returns at least one row. EXISTS operator simply tests whether the inner query returns any row. If it does, then the outer query proceeds. If not, the outer query does not execute, and the entire SQL statement returns nothing.

Syntax:

Select columns from tables where exists (subquery);

Retrieve the entire employee Names and Empno in all the departments.

```
SQL> SELECT empno, eName FROM Emp e
  WHERE EXISTS (SELECT 1
  FROM dept WHERE e.deptno = dept.deptno);
```

For each row from the EMP table, the condition is checked whether there exists a row in EMP table that has the employees.

- List all the employees detail who do not manage anyone

SQL> SELECT ENAME, JOB FROM EMP E

WHERE NOT EXISTS (SELECT * FROM EMP WHERE E.MGR=E.EMPNO);

Output will be the detail of all the employees such as their name and job.

Exercises

Use the FRIENDS table to answer the following questions.

1. Write a query that returns everyone in the database whose last name begins with M.
2. Write a query that returns everyone who lives in Illinois with a first name of AL.
3. Given two tables (PART1 and PART2) containing columns named PARTNO, how would you find out which part numbers are in both tables? Write the query.
4. What shorthand could you use instead of WHERE a >= 10 AND a <=30?
5. What will this query return?

```
SELECT FIRSTNAME
FROM FRIENDS
WHERE FIRSTNAME = 'AL'
AND LASTNAME = 'BULHER';
```

1. Using the FRIENDS table, write a query that returns the following:

<u>NAME</u>	<u>ST</u>
AL	FROM IL

2. Using the FRIENDS table, write a query that returns the following:

<u>NAME</u>	<u>PHONE</u>
MERRICK, BUD	300-555-6666
MAST, JD	381-555-6767
BULHER, FERRIS	345-555-3223

TRANSACTION, CONCURRENCY and RECOVERY Management

Transaction Management

- o A *transaction* is a collection of operations that performs a single logical function in a database application
- o Transaction-management component ensures that the database remains in a consistent (correct) state despite system failures (e.g., power failures and operating system crashes) and transaction failures.
- o Concurrency-control manager controls the interaction among the concurrent transactions, to ensure the consistency of the database.
- o System recovery - is the process of restoring a database to the correct state in the event of a failure.

Transaction

- ☒ A transaction is a part of program execution that accesses and updates various data items.
- ☒ A transaction can be defined as a group of tasks in which a single task is the minimum processing unit of work, which cannot be divided further.
- ☒ A transaction is a logical unit of work that contains one or more SQL statements.
- ☒ A transaction is an atomic unit (transaction either complete 0% or 100%).
- ☒ A database transaction must be atomic, meaning that it must be either entirely completed or aborted.

A *transaction* is an execution of a user program, and is seen by the DBMS as a series or list of actions. The actions that can be executed by a transaction include reads and writes of database objects, whereas actions in an ordinary program could involve user input, access to network devices, user interface drawing, etc.

Definition of some transaction Concepts & Terminologies

- (a) *Atomicity* means a transaction executes when all actions of the transaction are completed fully, or none are. This means there are no partial transactions (such as when half the actions complete and the other half do not).
- (b) *Consistency* involves beginning a transaction with a 'consistent' database, and finishing with a 'consistent' database. For example, in a bank database, money should never be "created" or "deleted" without an appropriate deposit or withdrawal. Every transaction should see a consistent database.
- (c) *Isolation* ensures that a transaction can run independently, without considering any side effects that other concurrently running transactions might have.
 1. When a database interleaves transaction actions for performance reasons, the database protects each transaction from the effects of other transactions.
- (d) *Durability* defines the persistence of committed data: once a transaction commits, the data should persist in the database even if the system crashes before the data is written to non-volatile storage.
- (e) A *schedule* is a series of (possibly overlapping) transactions.
- (f) *Abund write* is when a transaction writes to an object without ever reading the object.
- (g) A *dirty read* occurs when a transaction reads a database object that has been modified by another not-yet-committed transaction.
- (h) An *unrepeatable read* occurs when a transaction is unable to read the same object value more than once, even though the transaction has not modified the value. Suppose a transaction T2 changes the value of an object A that has been read by a transaction T1 while T1 is still in progress. If T1 tries to read the value of A again, it will get a different result, even though it has not modified A.
- (i) A *serializable schedule* over a set S of transactions is a schedule whose effect on any

consistent database instance is identical to that of some complete serial schedule over the set of committed transactions in S.

- (j) A *recoverable schedule* is one in which a transaction can commit only after all other transactions whose changes it has read have committed.
- (k) A schedule that *avoids-cascading-aborts* is one in which transactions only read the changes of committed transactions. Such a schedule is not only recoverable, aborting a transaction can be accomplished without cascading the abort to other transactions.
- (l) *Strict 2PL* is the most widely used locking protocol where:
 - 1) A transaction requests a shared/exclusive lock on the object before it reads/modifies the object.
 - 2) All locks held by a transaction are released when the transaction is completed.
- (m) The *phantom problem* is a situation where a transaction retrieves a collection of objects twice but sees different results, even though it does not modify any of these objects itself and follows the strict 2PL protocol. This problem usually arises in dynamic databases where a transaction cannot assume it has locked all objects of a given type (such as all sailors with rank 1; new sailors of rank 1 can be added by a second transaction after one transaction has locked all of the original ones).

If the set of database objects is fixed and only the values of objects can be changed, the phantom problem cannot occur since one cannot insert new objects into the database.

ACID properties of Transaction

Atomicity

- Either all operations of the transaction are properly reflected in the database or none are.
- Means either all the operations of a transaction are executed or not a single operation is executed.
- For example, consider transaction below: that transfer N50 from account A to account B:
 1. **read(A)**
 2. $A = A - 50$
 3. **write(A)**
 4. **read(B)**
 5. $B = B + 50$
 6. **write(B)**

In above transaction if N50 is deducted from account A then it must be added to account B.

Consistency

- Execution of a transaction in isolation preserves the consistency of the database.
- This means that our database must remain in consistent state after execution of any transaction.

In above example, the total of A and B must remain same before and after the execution of transaction.

Isolation

- Although multiple transactions may execute concurrently, each transaction must be unaware of other concurrently executing transactions.
- Intermediate transaction results must be hidden from other concurrently executed transactions.

In above example, once your transaction start from step one its result should not be access by any other transaction until last step (step 6) is completed.

Durability

- After a transaction completes successfully, the changes it has made to the database

persist, even if there are system failures.

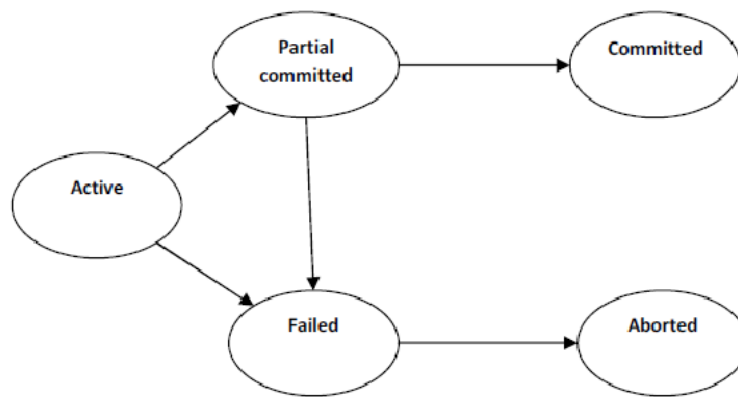
Once your transaction completed up to step 6 its result must be stored permanently. It should not be removed if system fails.

State Transition Diagram (or Transaction State Diagram).

Because failure of transaction may occur, transaction is broken up into states to handle various situations.

The following are the different states in transaction processing in database

Active
Partial committed
Failed
Aborted
Committed



Active

☒ This is the initial state. The transaction stays in this state while it is executing.

Partially Committed

☒ This is the state after the final statement of the transaction is executed.

☒ At this point failure is still possible since changes may have been only done in main memory, a hardware failure could still occur.

☒ The DBMS needs to write out enough information to disk so that, in case of a failure, the system could re-create the updates performed by the transaction once the system is brought back up.

☒ After it has written out all the necessary information, it is committed.

Failed

☒ After the discovery that normal execution can no longer proceed.

☒ Once a transaction cannot be completed, any changes that it made must be undone rolling it back.

Aborted

☒ The state after the transaction has been rolled back and the database has been restored to its state prior to the start of the transaction.

Committed

☒ The transaction enters in this state after successful completion of the transaction.

☒ We cannot abort or rollback a committed transaction.

Schedule

☒ A schedule is the chronological (sequential) order in which instructions are executed in a system.

⌘ A schedule for a set of transaction must consist of all the instruction of those transactions and must preserve the order in which the instructions appear in each individual transaction.

⌘ Example of schedule (Schedule 1)

T1	T2
read(A)	
A=A-50	
write(A)	
read(B)	
B= B+ 50	
write(B)	
	read(A)
	temp: A * 0.1
	A = A-temp
	write (A)
	read(B)
	B=B +temp
	write(B)

Serial schedule

- This is a Schedule that does not interleave the actions of different transactions.
- In schedule 1, all the instructions of T1 are grouped and run together. Then all the instructions of T2 are grouped and run together.
- Means schedule 2 will not start until all the instructions of schedule 1 are complete. This type of schedules is called serial schedule.

Interleaved schedule

- Schedule that interleave the actions of different transactions.
- Means schedule 2 can start before all instructions of schedule 1 are completed. This type of schedules is called interleaved schedule.

T1	T2
read(A)	
A:=A-50	
write(A)	
	read(A)
	temp: A * 0.1
	A:= A-temp
	write (A)
read(B)	
B:= B+ 50	
write(B)	
	read(B)
	B:=B +temp
	write(B)

Equivalent schedules

- Two schedules are equivalent schedule if the effect of executing the first schedule is identical (same) to the effect of executing the second schedule.
- We can also say that two schedule are equivalent schedule if the output of executing the

first schedule is identical (same) to the output of executing the second schedule.

Serializable schedule

- A schedule that is equivalent (in its outcome) to a serial schedule has the serializability property.
- Example of serializable schedule

Schedule 1			Schedule 2		
T1	T2	T3	T1	T2	T3
read(X) write(X)	read(Y) write(Y)	read(Z) write(Z)	read(X) write(X)	read(Y) write(Y)	read(Z) write(Z)

- In above example there are two schedules as schedule 1 and schedule 2.
- In schedule 1 and schedule 2 the order in which the instructions of transaction are executed is not the same but whatever the result we get is same. So, this is known as serializability of transaction.

Two phase commit protocol and working of two phase commit protocol.

Two phase commit protocol

- o The two-phase commit protocol provides an automatic recovery mechanism in case a system or media failure occurs during execution of the transaction.
- o The two phase commit protocol ensures that all participants perform the same action (either to commit or to roll back a transaction).
- o The two phase commit strategy is designed to ensure that either all the databases are updated or none of them, so that the databases remain synchronized.
- o In two phase commit protocol there is one node which is act as a coordinator and all other participating node are known as cohorts or participant.
- o Coordinator – the component that coordinates with all the participants.
- o Cohorts (Participants) – each individual node except coordinator are participant.
- o As the name suggests, the two-phase commit protocol involves two phases.
 1. The first phase is Commit-Request phase OR phase 1
 2. The second phase is Commit phase OR phase 2

Commit Request Phase (Obtaining Decision)

- To commit the transaction, the coordinator sends a request asking for “ready for commit” to each cohort.
- The coordinator waits until it has received a reply from all cohorts to “vote” on the request.
- Each participant votes by sending a message back to the coordinator as follows:
 - It votes YES if it is prepared to commit
 - It may vote NO for any reason if it cannot prepare the transaction due to a local failure.
 - It may delay in voting because cohort was busy with other work.

Commit Phase (Performing Decision)

- If the coordinator receives YES response from all cohorts, it decides to commit. The transaction is now officially committed. Otherwise, it either receives a NO response or gives up waiting for some cohort, so it decides to abort.
- The coordinator sends its decision to all participants (i.e., COMMIT or ABORT).
- Participants acknowledge receipt of commit or about by replying DONE.

System recovery

- ☒ Database recovery is the process of restoring a database to the correct state in the event of a failure.
- ☒ Database recovery is a service that is provided by the DBMS to ensure that the database is reliable and remain in consistent state in case of a failure.
- ☒ Restoring a physical backup means reconstructing it and making it available to the database server.
- ☒ To recover a restored backup, data is updated using redo command after the backup was taken.
- ☒ Database server such as SQL server or ORACLE server performs crash recovery and instance recovery automatically after an instance failure.
- ☒ In case of media failure, a database administrator (DBA) must initiate a recovery operation.
- ☒ Recovering a backup involves two distinct operations: rolling the backup forward to a more recent time by applying redo data and rolling back all changes made in uncommitted transactions to their original state.
- ☒ In general, recovery refers to the various operations involved in restoring, rolling forward and rolling back a backup.
- ☒ Backup and recovery refers to the various strategies and operations involved in protecting the database against data loss and reconstructing the database.

Log based recovery method.

Log based recovery

- ☒ The most widely used structure for recording database modification is the log.
- ☒ The log is a sequence of log records, recording all the update activities in the database.
- ☒ In short Transaction log is a journal or simply a data file, which contains history of all transaction performed and maintained on stable storage.
- ☒ Since the log contains a complete record of all database activity, the volume of data stored in the log may become unreasonable large.
- ☒ For log records to be useful for recovery from system and disk failures, the log must reside on stable storage.
- ☒ Log contains
 1. Start of transaction
 2. Transaction-id
 3. Record-id
 4. Type of operation (insert, update, delete)
 5. Old value, new value
 6. End of transaction that is committed or aborted.
- ☒ All such files are maintained by DBMS itself. Normally these are sequential files.
- ☒ Recovery has two factors Rollback (Undo) and Roll forward (Redo).
- ☒ When transaction T_i starts, it registers itself by writing a $\langle T_i \text{ start} \rangle$ log record
- ☒ Before T_i executes write(X), a log record $\langle T_i, X, V1, V2 \rangle$ is written, where $V1$ is the value of X before the write, and $V2$ is the value to be written to X .
- ☒ Log record notes that T_i has performed a write on data item X_j
- ☒ X_j had value $V1$ before the write, and will have value $V2$ after the write.

- ☒ When T_i finishes its last statement, the log record $\langle T_i \text{ commit} \rangle$ is written.
- ☒ Two approaches are used in log based recovery
 1. Deferred database modification
 2. Immediate database modification

Log based Recovery Techniques

- ☒ Once a failure occurs, DBMS retrieves the database using the back-up of database and transaction log. Various log based recovery techniques used by DBMS are as per below:
 1. Deferred Database Modification
 2. Immediate Database Modification
- ☒ Both of the techniques use transaction logs. These techniques are explained in following sub-sections.

Deferred Database Modification log based recovery method.

Concept

- ☒ Updates (changes) to the database are deferred (or postponed) until the transaction commits.
- ☒ During the execution of transaction, updates are recorded only in the transaction log and in buffers. After the transaction commits, these updates are recorded in the database.

When failure occurs

- ☒ If transaction has not committed, then it has not affected the database. And so, no need to do any undoing operations. Just restart the transaction.
- ☒ If transaction has committed, then, still, it may not have modified the database. And so, redo the updates of the transaction.

Transaction Log

- ☒ In this technique, transaction log is used in following ways:
 - ☒ Transaction T starts by writing $\langle T \text{ start} \rangle$ to the log.
 - ☒ Any update is recorded as $\langle T, X, V \rangle$, where V indicates new value for data item X . Here, no need to preserve old value of the changed data item. Also, V is not written to the X in database, but it is deferred.
 - ☒ Transaction T commits by writing $\langle T \text{ commit} \rangle$ to the log. Once this is entered in log, actual updates are recorded to the database.
 - ☒ If a transaction T aborts, the transaction log record is ignored, and no any updates are recorded to the database.

Example

- ☒ Consider the following two transactions, T_0 and T_1 given in figure, where T_0 executes before T_1 . Also consider that initial values for A , B and C are 500, 600 and 700 respectively.

Transaction – T_0	Transaction – T_1
Read (A)	Read (C)
$A = A - 100$	$C = C - 200$
Write (A)	Write (C)
Read (B)	
$B = B + 100$	
Write (B)	

- ☒ The following figure shows the transaction log for above two transactions at three different instances of time.

Time Instance (a)	Time Instance (b)	Time Instance (c)
<T ₀ start>	< T ₀ start>	< T ₀ start>
< T ₀ , A, 400>	< T ₀ , A, 400>	< T ₀ , A, 400>
< T ₀ , B, 700>	< T ₀ , B, 700>	< T ₀ , B, 700>
	< T ₀ commit>	< T ₀ commit>
	<T ₁ start>	< T ₁ start>
	<T ₁ , C, 500>	< T ₁ , C, 500>
		< T ₁ commit>

☒ If failure occurs in case of

1. No any REDO actions are required.
2. As Transaction T₀ has already committed, it must be redone.
3. As Transactions T₀ and T₁ have already committed, they must be redone.

System recovery procedure with Checkpoint record concept.

Problems with Deferred & Immediate Updates

☒ Searching the entire log is time-consuming.

☒ It is possible to redo transactions that have already been stored their updates to the database.

Checkpoint

☒ A point of synchronization between database and transaction log file.

☒ Specifies that any operations executed before this point are done correctly and stored safely.

☒ At this point, all the buffers are force-fully written to the secondary storage.

☒ Checkpoints are scheduled at predetermined time intervals

☒ Used to limit -

1. The size of transaction log file
2. Amount of searching, and
3. Subsequent processing that is required to carry out on the transaction log file.

When failure occurs

☒ Find out the nearest checkpoint.

☒ If transaction has already committed before this checkpoint, ignore it.

☒ If transaction is active at this point or after this point and has committed before failure, redo that transaction.

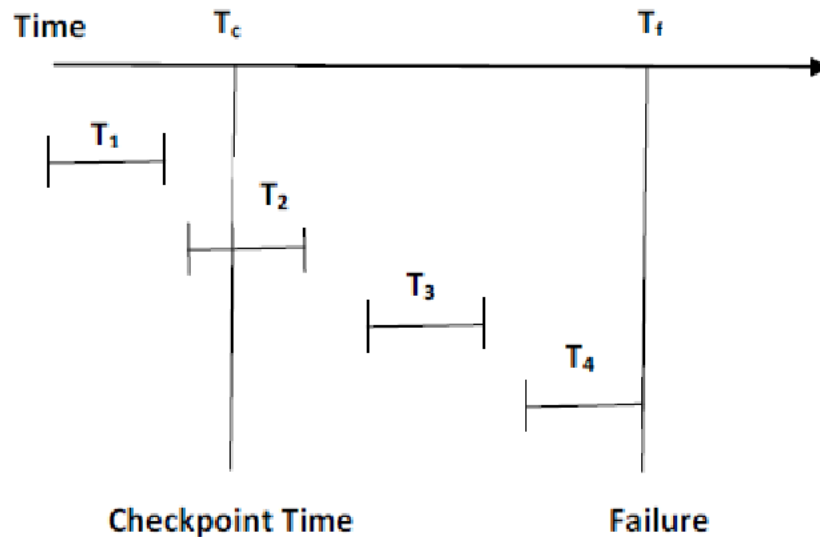
☒ If transaction is active at this point or after this point and has not committed, undo that transaction.

Example

☒ Consider the transactions given in following figure. Here, T_c indicates checkpoint, while T_f indicates failure time.

☒ Here, at failure time -

1. Ignore the transaction T₁ as it has already been committed before checkpoint.
2. Redo transaction T₂ and T₃ as they are active at/after checkpoint, but have committed before failure.
3. Undo transaction T₄ as it is active after checkpoint and has not committed.



Concurrency control? Why Concurrency control is needed?

Concurrency control

☒ The technique is used to protect data when multiple users are accessing (using) same data concurrently (at same time) is called concurrency control.

Concurrency control needed

☒ If transactions are executed serially, i.e., sequentially with no overlap in time, no transaction concurrency exists. However, if concurrent transactions with interleaving operations are allowed in an uncontrolled manner, some unexpected, undesirable result may occur. Here are some typical examples:

1. **The lost update problem:** This problem indicates that if two transactions T1 and T2 both read the same data and update it then effect of first update will be overwritten by the second update.
2. **The dirty read problem:** The dirty read arises when one transaction updates some item and then fails due to some reason. This updated item is retrieved by another transaction before it is changed back to the original value.
3. **The incorrect retrieval problem:** The inconsistent retrieval problem arises when one transaction retrieves data to use in some operation But before it can use this data another transaction update that data and commits. Through this change will be hidden from first transaction and it will continue to use previous retrieved data. This problem is also known as inconsistent analysis problem.

☒ Most high-performance transactional systems need to run transactions concurrently to meet their performance requirements. Thus, without concurrency control such systems can neither provide correct results nor maintain their databases consistent.

Define lock, locking and lock based protocol.

Lock

- ☒ A lock is a variable associated with data item to control concurrent access to that data item.
- ☒ Lock requests are made to concurrency-control manager.
- ☒ Transaction can proceed only after request is granted.

Locking

- ☒ One major problem in databases is concurrency.
- ☒ Concurrency problems arise when multiple users try to update or insert data into a database

table at the same time. Such concurrent updates can cause data to become corrupt or inconsistent.

☒ Locking is a strategy that is used to prevent such concurrent updates to data.

Lock based protocol

☒ A lock is a mechanism to control concurrent access to a data item

Data items can be locked in two modes :

1. Exclusive (X) mode. Data item can be both read as well as written. X-lock is requested using lock-X instruction.
2. Shared (S) mode. Data item can only be read. S-lock is requested using lock-S instruction.

☒ Lock requests are made to concurrency-control manager.

☒ Transaction can proceed only after request is granted.

☒ Lock-compatibility matrix

	S	X
S	TRUE	FALSE
X	FALSE	FALSE

☒ A transaction may be granted a lock on an item if the requested lock is compatible with locks already held on the item by other transactions

☒ Any number of transactions can hold shared locks on an item, but if any transaction holds an exclusive on the item no other transaction may hold any lock on the item.

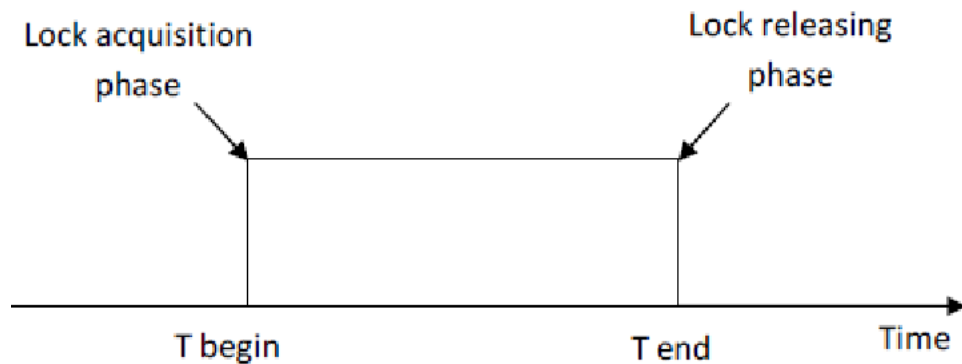
☒ If a lock cannot be granted, the requesting transaction is made to wait till all incompatible locks held by other transactions have been released. The lock is then granted.

T1	T2	Concurrency Control Manager
Lock-X(B)		
		Grant-X(B,T1)
Read(B)		
B=B-50		
Write(B)		
Unlock(B)		
	Lock-S(A)	
		Grant-S(A,T2)
	Read (A)	
	Unlock(A)	
	Lock-S(B)	
		Grant-S(B,T2)
	Read (B)	
	Unlock(B)	
	Display(A+B)	
Lock-X(A)		
		Grant-X(A,T1)
Read(A)		
A=A+50		
Write(A)		
Unlock(A)		

☒ This locking protocol divides transaction execution phase into three parts.

1. In the first part, when transaction starts executing, transaction seeks grant for locks it needs as it executes.
2. Second part is where the transaction acquires all locks and no other lock is required. Transaction keeps executing its operation.

3. As soon as the transaction releases its first lock, the third phase starts. In this phase a transaction cannot demand for any lock but only releases the acquired locks.



Methods to control concurrency

Methods to control concurrency (Mechanisms)

- ☒ **Optimistic** - Delay the checking of whether a transaction meets the isolation and other integrity rules (e.g., serializability and recoverability) until its end, without blocking any of its (read, write) operations and then abort a transaction to prevent the violation, if the desired rules are to be violated upon its commit. An aborted transaction is immediately restarted and re-executed, which incurs an obvious overhead. If not too many transactions are aborted, then being optimistic is usually a good strategy.
- ☒ **Pessimistic** - Block an operation of a transaction, if it may cause violation of the rules, until the possibility of violation disappears. Blocking operations is typically involved with performance reduction.
- ☒ **Semi-optimistic** - Block operations in some situations, if they may cause violation of some rules, and do not block in other situations while delaying rules checking (if needed) to transaction's end, as done with optimistic.

Methods to control concurrency (Methods)

- ☒ **Locking (Two-phase locking - 2PL)** - Controlling access to data by locks assigned to the data. Access of a transaction to a data item (database object) locked by another transaction may be blocked (depending on lock type and access operation type) until lock release.
- ☒ **Serialization graph checking (also called Serializability or Conflict or Precedence graph checking)** - Checking for cycles in the schedule's graph and breaking them by aborts
- ☒ **Timestamp ordering (TO)** - Assigning timestamps to transactions, and controlling or checking access to data by timestamp order.
- ☒ **Commitment ordering (Commit ordering or CO)** - Controlling or checking transactions' chronological order of commit events to be compatible with their respective precedence order.

Three problems due to concurrency and How the problems can be avoided.

Three problems due to concurrency

1. **The lost update problem:** This problem indicate that if two transactions T1 and T2 both read the same data and update it then effect of first update will be overwritten by the second update.
How to avoid: In example below, a transaction T2 must not update the data item (X) until the transaction T1 can commit data item (X).

T1	Time	T2
---	T0	---
Read X	T1	---
---	T2	Read X
Update X	T3	---
---	T4	Update X
---	T5	---

2. **The dirty read problem:** The dirty read arises when one transaction update some item and then fails due to some reason. This updated item is retrieved by another transaction before it is changed back to the original value.

T1	Time	T2
---	T0	---
---	T1	Update X
Read X	T2	---
---	T3	Rollback
---	T5	---

How to avoid: In above example a transaction T1 must not read the data item (X) until the transaction T2 can commit data item (X).

3. **The incorrect retrieval problem:** The inconsistent retrieval problem arises when one transaction retrieves data to use in some operation but before it can use this data another transaction updates that data and commits. Through this change will be hidden from first transaction and it will continue to use previous retrieved data. This problem is also known as inconsistent analysis problem.

Balance (A=200 B=250 C=150)		
T1	Time	T2
---	T0	---
Read (A) Sum $\leftarrow$ 200	T1	---
Read (B) Sum $\leftarrow$ Sum + 250 = 450	T2	---
---	T3	Read (C)
---	T4	Update (C) 150 $\rightarrow$ 150 - 50 = 100
---	T5	Read (A)
---	T6	Update (A) 200 $\rightarrow$ 200 + 50 = 250
---	T7	COMMIT
Read (C) Sum $\leftarrow$ Sum + 100 = 550	T8	---

Database Administrator

Coordinates all the activities of the database system; the database administrator has a good understanding of the enterprise's information resources and needs.

DBA - is Database Administrator.

☒ Database Administrator is a person in the organization who controls the design and the use of database.

Functions or Responsibilities of DBA are as under:

Schema Definition

- ☒ DBA defines the logical schema of the database.
- ☒ A schema refers to the overall logical structure of the database.
- ☒ According to this schema, database will be designed to store required data for an organization.

Storage Structure and Access Method Definition

- ☒ DBA decides how the data is to be represented in the database.
- ☒ Based on this, storage structure of the database and access methods of data is defined.

Defining Security and Integrity Constraints

- ☒ DBA decides various security and integrity constraints.
- ☒ DDL provides facilities to specifying such constraints.

Granting of Authorization for Data Access

- ☒ The DBA determines which user needs access to which part of the database.
- ☒ According to this, various types of authorizations (permissions) are granted to different users.
- ☒ This is required to prevent unauthorized access of a database.

Liaison with Users

- ☒ DBA is responsible to provide necessary data to user.
- ☒ User should be able to write the external schema, using DDL.

Assisting Application Programmers

- ☒ DBA provides assistance to application programmers to develop application programs.

Monitoring Performance

- ☒ The DBA monitors performance of the system.
- ☒ The DBA ensures that better performance is maintained by making change in physical or logical schema if required.

Backup and Recovery

- ☒ Database should not be lost or damaged.
- ☒ The task of DBA is to backing up the database on some storage devices such as DVD, CD or Magnetic Tape or remote servers.
- ☒ In case of failures, such as flood or virus attack, Database is recovered from this backup.

Differentiate the DA and DBA.

<i>DA (Data Administrator)</i>	<i>DBA (Database Administrator)</i>
The data administrator is a person in the organization who controls the data of the database.	The database administrator is a person in the organization who controls the design and the use of the database.
DA determines what data to be stored in database based on requirements of the organization.	DBA provides necessary technical support for implementing a database.
DA is involved more in the requirements gathering, analysis, and design phases.	DBA is involved more in the design, development, testing and operational phases.
DA is a manager or some senior level person in an organization who understands organizational requirements with respect to data.	DBA is a technical person having knowledge of database technology.
DA does not need to be a technical person, but any kind of knowledge about database technology can be more beneficiary.	DBA does not need to be a business person, but any kind of knowledge about a functionality of an organization can be more beneficiary.
DA is a business focused person, but, he/she should understand more about the database technology.	DBA is a technically focused person, but, he/she should understand more about the business to administer the databases effectively.